

SZEGEDI TUDOMÁNYEGYETEM

Természettudományi és Informatikai Kar

Szoftverfejlesztés Tanszék

Szakdolgozat

Fuvarmegosztó Web Applikáció

Ride Sharing Web Application

Vrabély Gellért László

Programtervező informatikus BSc.

Témavezető: Dr. Pengő Edit, adjunktus

Szeged

2026.

Feladatkiírás

A hallgató feladata egy olyan fuvarmegosztó webalkalmazás tervezése és teljes körű fejlesztése, amely a hagyományos telekocsizás alapelveit helyezi előtérbe. A projekt fókuszában egy olyan platform létrehozása áll, amely automatizált módon számítja ki az utazások költségeit, ezzel garantálva az igazságos tehermegosztást.

A megvalósítás során a korszerű webfejlesztési alapelveket szem előtt tartva kell a feladatot elvégezni. Egy olyan többretegű létrehozása alkalmazásarchitektúra a cél, mely elemei között a kommunikációt a szabványos http protokoll végzi. A rendszernek integrálnia kell a biztonságos felhasználókezelést, a stabil adattárolást, valamint a külső szolgáltatásokkal való kommunikációt. Az alkalmazás websocket létrehozásával valósítson meg valamely kontextusban élő adatkommunikációt.

Tartalmi összefoglaló

Téma megnevezése

Fuvarmegosztó web applikáció

A megadott feladat megfogalmazása

Egy olyan **web applikáció** elkészítése, mely a **fuvarmegosztást**, mint szolgáltatást **népszerűsíti**. Az alkalmazás legyen hatékonyabb és egyedi a hasonló szolgáltatásokhoz képest.

Megoldási mód

Egy **Angular keretrendszerben** megírt weboldalt készítettem, melyet egy **Java Spring Boot** szerver szolgál ki erőforrásokkal **REST API végpontokon** keresztül. Mindkét réteget a Keycloak alkalmazás autentikálja.

Alkalmazott eszközök, módszerek

Az **IntelliJ IDEA** fejlesztői környezetet használtam a fejlesztés során. Az oldal programozása alatt végig betartottam a **clean code** és **DRY** (Don't Repeat Yourself) elveit, például minden többször felmerülő viselkedés ki van szervezve segédmétódusba. Szintén ezeket az elveket szem előtt tartva, a szerver oldalon **Controller-Service-Repository**, a frontend oldalon pedig a **Component-Service** architektúrális mintát alkalmaztam. A buildelés és függőségkezelés könnyítése érdekében **node package manager**-t és **maven**-t alkalmaztam. Az alkalmazás szállíthatóságához, és a **Keycloak** futtatásához a **Docker** konténerizációs alkalmazást használtam. Ilyen konténerekben fut a szerver és a Keycloak **PostgreSQL adatbázisa** is.

Elért eredmények

Az alkalmazás visszaszorítja a nyereszkedő sofőröket, így hatékonyabbá teszi a fuvarmegosztás szolgáltatást. Felhasználóbarát kinézet és kényelmi funkciók által gyarapítja a szolgáltatást igénybe vevők számát.

Kulcsszavak

fuvarmegosztás, objektív költségszámítás, reszponzivitás, helymeghatározás

Tartalomjegyzék

Feladatkiírás.....	2
Tartalmi összefoglaló.....	3
Téma megnevezése	3
A megadott feladat megfogalmazása.....	3
Megoldási mód	3
Alkalmazott eszközök, módszerek	3
Elért eredmények	3
Kulcsszavak	3
Tartalomjegyzék	4
1 Bevezetés.....	6
2 Architektúra.....	7
2.1 Konténerizáció	8
2.2 Frontend	10
2.3 Backend.....	11
2.4 Security – Keycloak	13
2.4.1 Frontend	13
2.4.2 Backend.....	14
2.4.3 Futtatás	14
2.5 Adatbázis	14
3 Főbb funkciók ismertetése	16
3.1 Sötét/Világos mód.....	16
3.2 Helymeghatározás térképen	17
3.3 Fuvarmegosztás.....	18
3.4 Intelligens fuvar szűrés és lapozás	19
3.5 Utas jóváhagyás.....	20

3.6	Kommunikáció	21
3.7	Értékelési rendszer	22
3.8	Értesítési rendszer	24
4	Főbb funkciók implementálása	25
4.1	Sötét/Világos mód	25
4.2	Helymeghatározás térképen	25
4.3	Fuvarmegosztás	26
4.4	Intelligens fuvar szűrés és lapozás	27
4.5	Utas jóváhagyás.....	29
4.6	Kommunikáció	30
4.7	Értékelési rendszer	31
4.8	Értesítési rendszer	33
5	Globális hiba kezelés.....	35
6	Út árának számítása — PriceCalculatorService	37
6.1	Tervezett út hossza	37
6.2	Férőhelyek, üzemanyag-fogyasztás és gyártási év.....	38
6.3	Kilométerenkénti értékcsökkenés	39
6.4	Végső kalkuláció	40
7	Piaci összehasonlítás	41
8	Befejezés	42
9	Irodalomjegyzék.....	43
10	Nyilatkozat	44
11	Köszönetnyilvánítás	45

1 Bevezetés

A **fuvarmegosztás** egy olyan közlekedési forma, amikor egy magánszemély megosztja a gépjárművében lévő szabad helyeket azonos irányba tartó utasokkal. Szimulációs kutatások bizonyítják, hogy a fuvarmegosztás elterjedése átlagosan **6.3%-kal csökkentheti** a városi közlekedés **CO₂ kibocsátását** [1]. A hazai piacon működő megoldások eltávolodtak a hagyományos fuvarmegosztási modelltől, és működésük fókuszába a profitmaximalizálás került. Pedig a kihasználatlan ülőhelyek száma drasztikusan magas: az Európai Bizottság szerint a világ útjain jelenleg közlekedő körülbelül 1 milliárd autó közül 740 milliót csak egy személy használ [2].

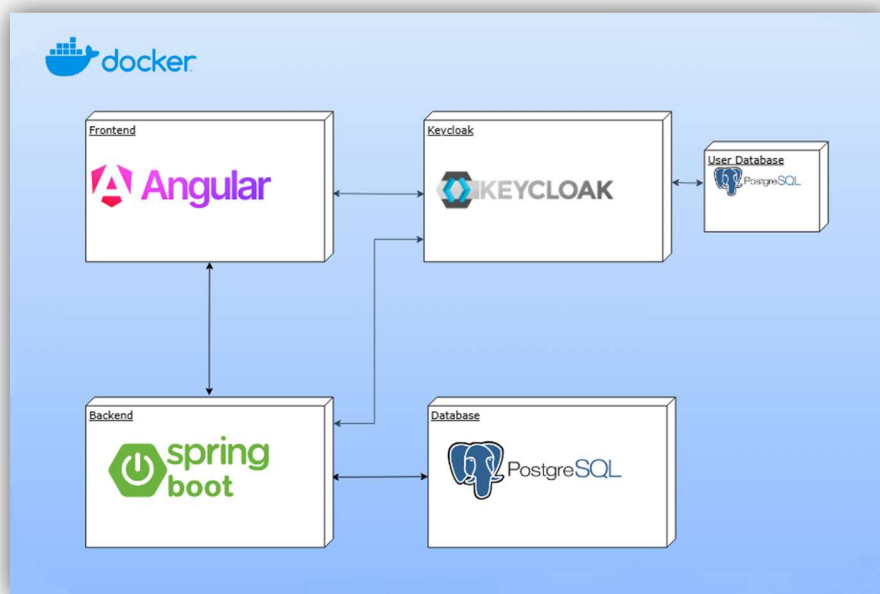
A tartalmi összefoglalóban felvázolt projekt célja a hagyományos értelemben vett telekocsizás visszaállítása, aminek eredeti koncepciója, hogy a sofőrök már egyébként is tervezett útjaikhoz keresnek utasokat. A projekt sikere érdekében elengedhetetlen egy jól kezelhető, **felhasználóbarát platform**, amely képes **objektív útiköltséget számítani**. Ennek megvalósítására egy olyan többretegű webes alkalmazást fejlesztettem ki, amely szoftverszinten korlátozza a profitorientált magatartást. A rendszer neutrális útiköltséget számít külső szolgáltatások bevonásával, és a felhasználó által megadott paraméterekből. Ez a mechanizmus elveszi a sofőrök kezéből az árazás feletti kontrollt, így garantálja az egyenlő költségmegosztást.

A tisztességes anyagi feltételek mellett a fuvarmegosztó közösség alapját a **kommunikáció** és az erre épülő **bizalom** adja. Ezt felismerve az alkalmazásom kétirányú **értékelési rendszert**, valamint egy **valós-idejű chat** modult is megvalósít, amely a hagyományos **HTTP protokoll** mellett **Websocket** technológiára épül. A rendszer a redundáns adatok tárolása elkerülése végett rugalmasan kezeli a felhasználók szerepköreit. Emiatt a felhasználó egy és ugyanazon profillal képes utas és sofőr is lenni.

A mai modern szoftverfejlesztésben a felhasználói élmény (UX) elengedhetetlen sikertényező. Emiatt az alkalmazásban korszerű elrendezési technikákat (**Grid**, **Flexbox**), és design elveket követtem, amelyeknek köszönhetően bármely eszközön letisztult megjelenés fogadja a klienst. Az intuitív használatot olyan funkciók segítik elő, mint a **dinamikus gépjármű-kereső**, IP-cím vagy GPS alapú helymeghatározás **Geolocation API** segítségével, valamint a háttérben futó **e-mail értesítési rendszer**, amely naprakészen tartja a végfelhasználókat.

2 Architektúra

Az applikáció 4 elkülönült rétegből épül fel, mint a **frontend** (a kinézet), **backend** (az üzleti logika), az **adatbázis** (adattár, ami kiszolgálja a backendet) és a **biztonsági réteg** (amely autentikálja a backendet és a frontendet). A fejlesztés hatékonysága és a könnyű szállítás¹ érdekében a **Docker** konténerizációs szolgáltatást használtam, ez az, ami össze fogja a 4 réteget egy egymással kommunikáló szoftvercsomaggá. A frontend **Angular 20**, a backend pedig **Java Spring Boot 3.5** keretrendszert használ a felhasználói élmény növelése és a komplexitás elfedése érdekében. A backendet kiszolgáló adatbázis egy **Postgres 17** példány². A klienst és szerveret is autentikáló biztonsági réteg egy **Keycloak** nevű open-source alkalmazás, mely szintén konténerizált környezetben fut — a fejlesztés könnyebbsége érdekében — az alkalmazás többi részével egy környezetben. A rendszer felépítését a **2.1-es ábra** szemlélteti.



2.1 Ábra, rendszerarchitektúra vázlat

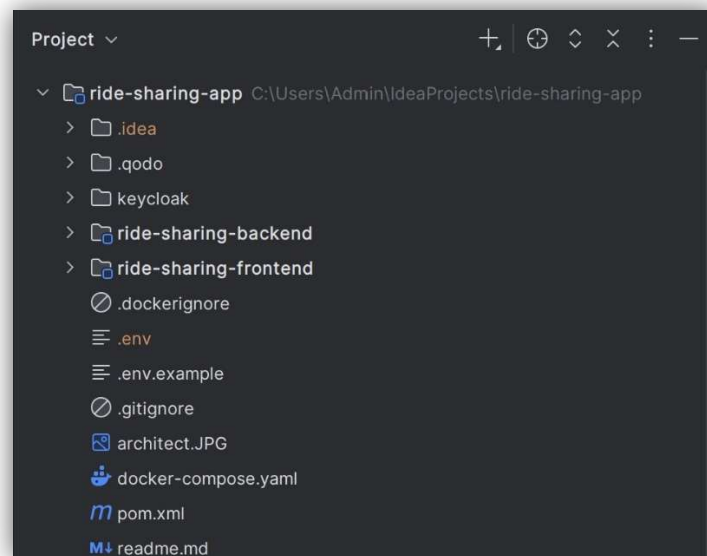
A fejlesztés során professzionális, monorepóra³ illeszkedő mappastruktúrát alakítottam ki, amit a **2.2-es ábra** mutat be. A backend és frontend két külön modulként viselkedik, architektúra szempontjából teljes mértékben szét vannak választva. Mindkét mappa, illetve a

¹ **Szállítás:** Informatikai értelemben a szállítás, más néven deploy azt a fogalmat jelenti, amikor egy szoftvert éles környezetbe helyezzük.

² **Példány:** Szoftverfejlesztés világában példánynak hívunk egy élő alkalmazást vagy objektumot. Ez a fogalom konténerizációs térben kifejezetten elterjedt.

³ **Monorepo:** Olyan rendezési stratégia, amikor több, akár teljesen külön álló modult tartunk egy tárolóban.

gyökérkönyvtár is tartalmaz egy pom.xml fájlt, ezek konfigurálást, függőségeket tartalmaznak, amiket a maven projekt menedzser eszköz kezel. A projekt szerkezetét a mutatja be.



2.2 Ábra, mappastruktúra

2.1 Konténerizáció

A konténerizáció az a folyamat amikor az alkalmazások és függőségeik egy konténernek nevezett egységbe vannak csomagolva. Ez lehetővé teszi, hogy **operációs rendszertől független** futtatható legyen az alkalmazás a számítógépen.

A dolgozatomban a **Docker** nevű széles körben elterjedt szoftvert használtam konténerizációra. Egyebek közt számomra amiatt is elengedhetetlen volt ennek az eszköznek a használata, mert egy harmadik féltől származó alkalmazást használtam a biztonsági réteghez. Ezt éles környezetben egy külön szerveren kellene futtatni. Lokális környezetben legegyszerűbb volt a Docker segítségével elindítani, és egy hálózatba helyezni az alkalmazás többi részével.

Mind a backend, mind a frontend mappában található egy **Dockerfile**, ez egy egyszerű szöveges dokumentum, ami a Docker számára tartalmaz információkat arról, hogyan építse fel a **Docker image**⁴-et. A legtöbb image egy másikra épül rá, tehát van egy alapja (base image), így az alap függőségeit, előnyeit már tartalmazza.

⁴ **Docker image**: Egy önálló, futtatható csomag, ami tartalmazza a könyvtárakat és függőségeket.

A **2.3-as kódrészlet** mutatja be a **frontend Dockerfile-t**, az applikáció ezen része egy **node:20-slim** nevű image-re épül, ez egy könnyűsúlyú, lecsupaszított Node.js szerver, ami fejlesztői környezetben tökéletesen megfelel a kliens kiszolgálására, és az Angular keretrendszer parancsainak futtatására. A Dockerfile először csak a függőségeket tartalmazó fájlt használja, mert ebben ritkán van változás, és ha nem lát változást, ezt a folyamatot könnyedén **gyorsítótárazza**. Csak ezután másolja le a teljes forráskódot, és indítja el az Angular szerveret. Fontos megjegyezni, hogy éles környezetben ez általában nem Node.js és Angular szerverrel van futtatva, hanem statikus fájlokra fordítás után, egy webszerverrel szolgáltatjuk ki.

A **backend** az **eclipse-temurin:21-jdk** imagere épül, ez egy teljes Java Fejlesztői Eszköz. Fontos, hogy a backend Dockerfile, ami a **2.4-es kódrészleten** látható, feltételezi, hogy a **backend/target** mappa alatt már létezik

a **.jar** futtatható Java csomag. Ami azt jelenti, hogy a fejlesztőnek a maven csomagkezelővel előtte el kell készítenie ezt, a *mvn clean package* paranccsal. Ez a megoldás azért jó, mert ha később egy **CI/CD pipeline**⁵-t használunk, akkor ezt a maven parancsot

```
FROM node:20-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 4200
CMD ["npm", "start"]
```

```
FROM eclipse-temurin:21-jdk
WORKDIR /app
COPY target/*.jar app.jar
ENTRYPOINT ["java", "-jar",
"app.jar"]
```

egyszerűen beintegráljuk a szekvenciájába.

2.3 Kódrészlet, frontend Dockerfile

2.4 Kódrészlet, backend Dockerfile

Az egyszerű indításban, és a Dockerfile-ok és egyéb szolgáltatások (Adatbázis, Biztonság) egységbezárásában a **docker-compose.yaml** fájl segít a Dockernek. Ez egy olyan yaml struktúrában megírt állomány, amely által egyéb konfigurációk (például: környezeti változók, hálózati portok, újraindítási feltételek) állíthatók be. Emellett egy belső hálózatot hoz létre, amin keresztül az úgynevezett service-ek (docker compose-ban futó konténerek) tudnak egymással kommunikálni. Továbbá, a **Docker Engine** ezt a fájlt felolvasva képes egyszerre az összes konténert elindítani, egyben futtatni őket.

⁵ **Pipeline:** Kóddal automatizált munkafolyamat.

2.2 Frontend

A felhasználói felület implementálása során — architektúrális szempontból — az **Angular** konvencióit követtem. A frontend mappa további két mappára (*public*, *src*) és egyéb fájlokra osztozik. A *public* mappa statikus fájlok tárolására szolgál, az alkalmazás jelenlegi állapotában csak képek vannak benne.

Az *src* mappában forráskódhoz köthető fájlok és könyvtárak vannak: A *components* mappában található a komponensek, **HTML**, **SCSS** és **TypeScript** fájlok összessége, amelyek általában egy adott oldalt vagy egy megjelenő felületet (pl.: chat-ablak) fognak össze. A *model* mappában található az adattároló interfészek, ezek a **szerverrel való kommunikálásra** létrehozott entitások, amelyek JSON formátummá/formátumból vannak szerializálva⁶/deszerializálva⁷. Végezetül pedig a *service* mappában a kommunikációért felelős singleton⁸ TypeScript osztályok vannak.

Kiemelendő egy fájl a frontend gyökérkönyvtára alatt, mely a kérések átirányításáért felel. Ez a fájl a *proxy.conf.json*, ami a CORS (Cross-origin Resource Sharing) hiba elkerülése miatt elengedhetetlen. CORS hiba akkor keletkezik, ha egy nem engedélyezett domain adatokat szeretne lekérni egy másik eszközről. Mivel ez a böngészők beépített biztonsági mechanizmusa, így a *localhost:4200*-on futó frontend nem tudná meghívni a *localhost:8080*-on futó backendet. A proxy konfiguráció áthidalja ezt a problémát, a frontend kéréseit saját lokális szerveren keresztül továbbítja a backend felé, így a böngésző nem érzékel keresztirányú hívást [3].

A frontend egy **SPA** (Single Page Application), ami egy web dokumentumot tölt be, és ennek a tartalmát frissíti JavaScript segítségével. Ennek a logikáját maga az Angular keretrendszer valósítja meg, így a fejlesztőtől teljesen el van rejtve a komplexitás. Az Angular alkalmazás alapkövei a frontend gyökérkönyvtárában található *main.ts*, *app.config.ts*, *app.routes.ts* fájlok. Ezek felelnek — felsorolás sorrendje alapján — az alkalmazás belépéséért, globális konfigurációért és az útvonalválasztásért.

A letisztult megjelenés és felhasználói élmény érdekében a keretrendszerhez kettő meghatározó könyvtárat használtam: **Tailwind CSS** és **PrimeNG**. Az interaktív elemekhez, mint dátumválasztó, legördülő lista, számbeviteli mezők, a PrimeNG előre megírt

⁶ **Szerializálás:** Az a folyamat, amikor egy programozási nyelv számára érthető, memóriában tárolt komplex adatstruktúrát, kommunikációra vagy fájlba mentésre alkalmas formátumúvá alakítunk (JSON, XML).

⁷ **Deszerializálás:** Az a folyamat, amikor eszközön tárolt vagy kommunikációs csatornáról érkező adatot, egy programozási nyelv számára érthető formátumúvá alakítunk.

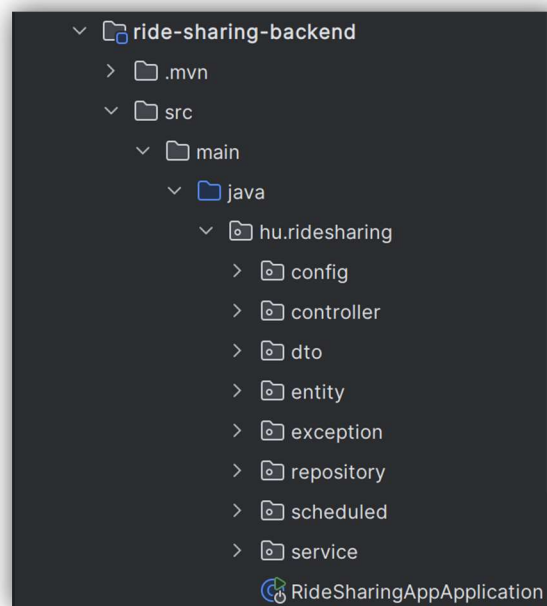
⁸ **Singleton:** Egy olyan tervezési minta, amely biztosítja, hogy az osztályból csak egy példány létezzen.

komponenseit alkalmaztam. Ezek előnye, hogy konfigurálhatók az Angular modern Input, vagy InputSignal mechanizmussal. A stílusok finomításához és a töréspontok beállításához a Tailwind CSS könyvtárat használtam. Ennek használatával CSS media query beállítások nélkül lehet szabályozni az elrendezést, így lényegesen átláthatóbb, könnyebben értelmezhető kódot kaptam.

2.3 Backend

A szerveroldali kód esetében a mappastruktúra a **Java Spring Boot** konvencióit valósítja meg. Az *src / main* mappa alatt kettő másik mappa található, *java* és *resources*.

A *resources* mappa alatt található az *application.properties* konfigurációs fájl, ahol kulcs-érték párokban állítunk be szenzitív adatokat, mint API kulcsok, adatbázis jelszó, vagy egyéb konfigurálható értékek.



2.5 Ábra, backend forráskód mappastruktúra

A 2.5-ös ábrán a **java** mappa struktúrája látszik, itt található a forráskód: *config* mappában vannak tárolva a konfigurációs fájlok, ahol a Websocket és a Security van bekonfigurálva. A *controller* mappában *@RestController* annotációval ellátott osztályok vannak, amelyek a HTTP protokollon való kommunikációért felelősek, itt fogadjuk a frontend kéréseit, és küldünk rá választ. A *dto* mappában a szerializációra/deszerializációra alkalmas osztályok vannak, angol nevén data transfer objectek. Az *entity* mappában vannak tárolva az adatbázis entitások, itt **JPA (Java Persistence API)** bevonásával vannak megírva Java

osztályok. Ezeket egy ORM keretrendszer képezi le **PostgreSQL** adatbázis táblákra. Így a szinkronizáció a két formátum között át van hidalva. Az *exception* mappában hiba-osztályok vannak tárolva, illetve egy központi hibakezelő, ami a válasz megfelelő **HTTP státusz kódja** és üzenete beállításáért felelős hiba esetén. A *repository* mappa tartalmazza az adatbázissal kommunikáló interfészeket, ezek a **JpaRepository** interfészekből öröklődnek, melyek előre definiált **CRUD**⁹ parancsokat, illetve lapozási és rendezési módosításokat implementálnak. A *scheduled* mappa tartalma egy időzített automatikus email küldő rendszer. Végezetül pedig a *service* mappa, ahol *@Service* annotációval jelölt osztályok vannak, amelyek üzleti logikát valósítanak meg, és szolgálják ki a Controller objektumokat közvetlenül. Az utolsó elem egy osztály, ami az egész alkalmazás belépési pontja, a **RideSharingAppApplication**.

A backend egy **RESTful API architektúrát** valósít meg. Ez jól látható a *@RestController* annotáció használatából, vagy a JSON formátumban adott válaszokból is. A RESTful API egyik fő ismérve, hogy állapotmentes (stateless). Ez azt jelenti, hogy minden kliens kérését egymástól függetlenül bármely sorrendben képesek teljesíteni.

A szerver külső rendszerekkel, úgynevezett **API**-kal kommunikál a felhasználói élmény javítása és a költségek számítása érdekében. A rendszer három ilyen külső szolgáltatást használ, az **OpenWeatherMap** [4] városból koordináta és koordinátából város meghatározására, a **OpenRouteService** [5] úthossz számításra két koordináta között, és a **RapidApi** [6], amely a gépjármű márkát, modellt, pontos típust, székek számát, fogyasztást és üzemanyag típusát hivatott válaszként adni. Ezen információk által a felhasználó könnyedén megosztja a fuvarját az oldalon, ami automatikusan árat szab az utasok számára, ezzel elkerülve a sofőr profitmaximalizálását.

A külső szolgáltatásokkal való kommunikáció során HTTP kérést küldünk, ezekhez általában a fejlécbe **API kulcsot** kell megadnunk. Mivel a kódban készítjük el a kérést, a kódnak ismernie kell ennek a kulcsnak az értékét. Azonban, ilyen kritikus adatokat nem ajánlott a forráskódban tárolni, ezért ezek a már említett *application.properties* fájlból jönnek, ahonnan a Spring futásidőben felolvassa, és beinjektálja a *@Value* annotáció alkalmazásával. A projekt repozitóriuma publikus GitHubon, emiatt a legjobb, ha ezek a szenzitív adatok csak a lokális fejlesztői gépen vannak tárolva. Ennek segítségére van egy *.env* és egy *.env.example* fájl, ezek közül csak az utóbbi verziókezelte fájl, előbbi expliciten hozzá van adva a *.gitignore* fájlhoz. A *.env.example*-ben szenzitív adatok kulcsai vannak deklarálva, értékei nem. Így az érzékeny

⁹ **CRUD**: A négy alapvető SQL parancs rövidítése: Create, Read, Update, Delete

adatok biztonságban vannak tárolva a fejlesztő lokális állományaiban. A Docker a *docker-compose* fájlban deklarált kulcsok értékeit automatikusan felolvassa a *.env* fájlból, és beállítja a futó konténer operációs rendszerének környezeti változóiként. A **2.6-os kódrészletben** az látható, ahogy az *application.properties* fájlban string interpolációval hivatkozunk ezen környezeti változók kulcsaira, és a Spring beinjektálja az értékeket.

```
open.route.api.key=${OPEN_ROUTE_API_KEY}
geocoding.api.key=${GEOCODING_API_KEY}
rapid.api.url=${RAPID_API_URL}
rapid.api.key=${RAPID_API_KEY}
```

2.6 Kódrészlet, hivatkozás kulcsértékekre string interpolációval

2.4 Security – Keycloak

A biztonsági réteg megvalósításában a Keycloak volt segítségemre, ez egy open-source **autentikációs/authorizációs** alkalmazás. OpenID Connect protokollt használ, ami a mai sztenderdekot követve **JSON Web Token** (JWT) alkalmaz. A JSON Web Token a Keycloak aláírja a saját privát kulcsával, majd bármely alkalmazás (akár frontend, akár backend) ellenőrizni tudja a token hitelességét a Keycloak publikus kulcsával. [7]

Az alkalmazás a szerveren és a felhasználói felületen is alkalmazza ezt a szolgáltatást, architektúra szempontjából a következő a feladatuk:

2.4.1 Frontend

A frontend belépteti a felhasználót a rendszerbe, és a token elmenti: Amikor a felhasználó rákattint a belépés gombra, az Angular átirányítja a Dockerben futtatott Keycloak példány bejelentkezési felületére (itt van lehetőség a regisztrációra is). Sikeres bejelentkezés után az applikáció egy access token-t kap, amit eltárol a localStorage-ban. Ez az access token csak rövid ideig érvényes, viszont a frontend képes ezt a token-t frissíteni egy refresh token-nel, mindezt anélkül, hogy a felhasználót újból bejelentkeztetné. Az alkalmazás egy úgynevezett **HttpInterceptor** segítségével minden kimenő kérés fejlécébe beilleszti ezt a token-t, *Authorization* kulccsal és *Bearer <token>* értékkel. Emellett van egy authorizációs guard implementálva, amely bizonyos oldalakat véd „vendég” felhasználóktól, tehát bejelentkezésre kényszeríti őket, csak bejelentkezett felhasználóknak ad elérést. Ilyen guard-dal védett például az üzenetfelületet megvalósító komponens, hisz a perzisztenciához elengedhetetlen a felhasználónév tárolása.

2.4.2 Backend

A backend egy **Resource Server**¹⁰ szerepét tölti be a biztonsági rétegen. A kliens oldalról érkező kérés fejlécében szerepel egy **JWT token**, ezt a **Spring Security** segítségével elkapjuk, majd dekódoljuk: ellenőrzi az aláírást és a lejáratot. Ha érvényes a token, akkor a Spring kinyeri belőle a kritikus információkat, mint például username, role, és elmenti saját Bean osztályba¹¹. Ennek a konfigurálásáért a *SecurityFilterChain* Bean és az **application.properties** fájl felel [8].

2.4.3 Futtatás

A **Keycloak** az én megvalósításomban egy **Docker konténerben** fut, egy saját adatbázissal együtt. Az alkalmazás repository-jában a keycloak mappa alatt található egy *ride-sharing-realm.json* fájl, amely biztosítja, hogy az alapvető realmek¹², kliensek inicializálva legyenek a szoftvert futtató számítógépen, ennek beimportálása elengedhetetlen az alkalmazás működéséhez.

2.5 Adatbázis

Az adattárolásért egy **PostgreSQL 17** relációs adatbázist használok. Az entitásokat, kapcsolatokat és alapvető CRUD műveleteket a **Spring Data JPA** (Java Persistence API) fordítja SQL-re. Az adatbázis konfigurálásánál meg kell adni a *spring.datasource.url*-nak a **JDBC**¹³ URL-t. Ebből a Spring automatikusan kiolvassa a dialektust, ami jelen esetben a Postgres, így képes a JPA lefordítani a Java-ban írt kódot egy PostgreSQL relációs-adatbázisnak.

Az alkalmazás üzleti logikájához elengedhetetlenek a jól megtervezett entitások, és ezek kapcsolatai:

- **User:** Központi entitás, amely adatait a Keycloak-ból szerezzük, egy User lehet sofőr vagy utas is, de ugyanazon az úton csak az egyik.
- **Journey (utazás):** A fuvarokat tároló entitás, tartalmazza az indulási, érkezési adatokat, sofőrt, szabad helyeket, egy főre jutó árat és a gépjármű márkáját.

¹⁰ **Resource Server:** (védett) Adatokat tároló/hosztoló API, amely validálja a kientől érkező access tokeneket.

¹¹ **Bean osztály:** Olyan osztálypéldány, mely életciklusát a Spring keretrendszer kezeli.

¹² **Realm:** egy olyan elkülönített részleg, ahol felhasználókat, applikációkat, szerepköröket menedzselünk

¹³ **JDBC:** Java Database Connectivity egy Java API ami lehetővé teszi a csatlakozást, lekérdezést és adatmanipulációt a relációs adatbázisokkal

- **Rating:** Értékelés a felhasználók között, két típusa lehetséges:
 - *PASSENGER_TO_DRIVER*
 - *DRIVER_TO_PASSENGER*
- **ChatMessage:** A felhasználók közötti üzenetek perzisztenciájára szolgál: eltárolja ki, kinek, mit és mikor küldött.
- **Route:** Két város közötti távolságot és utazás hosszát tárolja el, emellett elmenti a két város koordinátáit, hogy újra felhasználhatóak legyenek.

Az adatmodell tervezése során több entitás is kapott **összetett kulcsot**, mert egyértelműen meghatározzák őket valamely adattagjaik. Erre kiváló példa a **Rating**, ebben az esetben azért fontos az összetett kulcs, mert bár egy felhasználó több értékelést is adhat és többször értékelheti ugyanazt a személyt, egy **utazáson belül csak egyszer értékelhet**. Az összetett kulcs tehát tartalmazza a *Journey* kulcsát, az értékelő és értékelt (személy) kulcsát, és az értékelés irányát.

Az alkalmazás egy dedikált **entitáscsoportot** tart fent a külső autóval kapcsolatos adatok tárolására, ez az *entity/car* mappa alatt található. Itt vannak eltárolva a RapidAPI-től érkező autóadatok, amiket az alkalmazás gyorsítótárazni képes. Ezek a *Car*, *CarModel*, *CarGeneration*, *CarTrim*. Különlegességét az adja, hogy struktúrája **követi a külső API válaszformátumát**, így könnyedén lekérhető egyik adatból a további adat. Ez a funkció kritikus annak érdekében, hogy az utazás meghirdetésénél a sofőr dinamikus listából tudja kiválasztani az autó adatait.

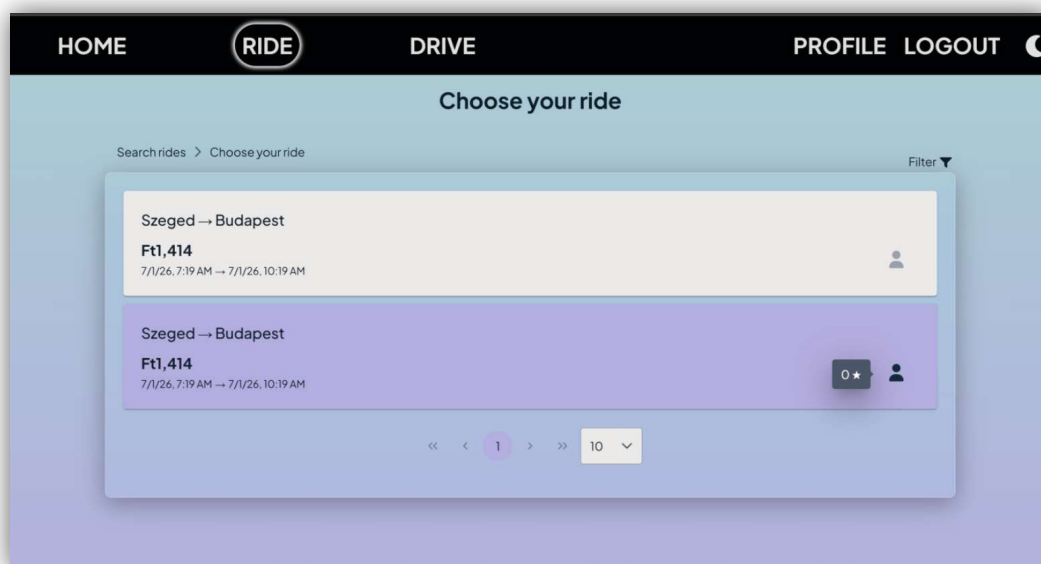
3 Főbb funkciók ismertetése

A fejlesztés során kiemelt figyelmet fordítottam a felhasználói élményre, valamint a kliens és a szerver közötti hatékony interakciókra. E célok elérése érdekében modern, iparági sztenderdeknek megfelelő megoldásokat alkalmaztam. Az alábbi **nyolc** pontban részletezem az ehhez elengedhetetlen legfőbb funkciókat.

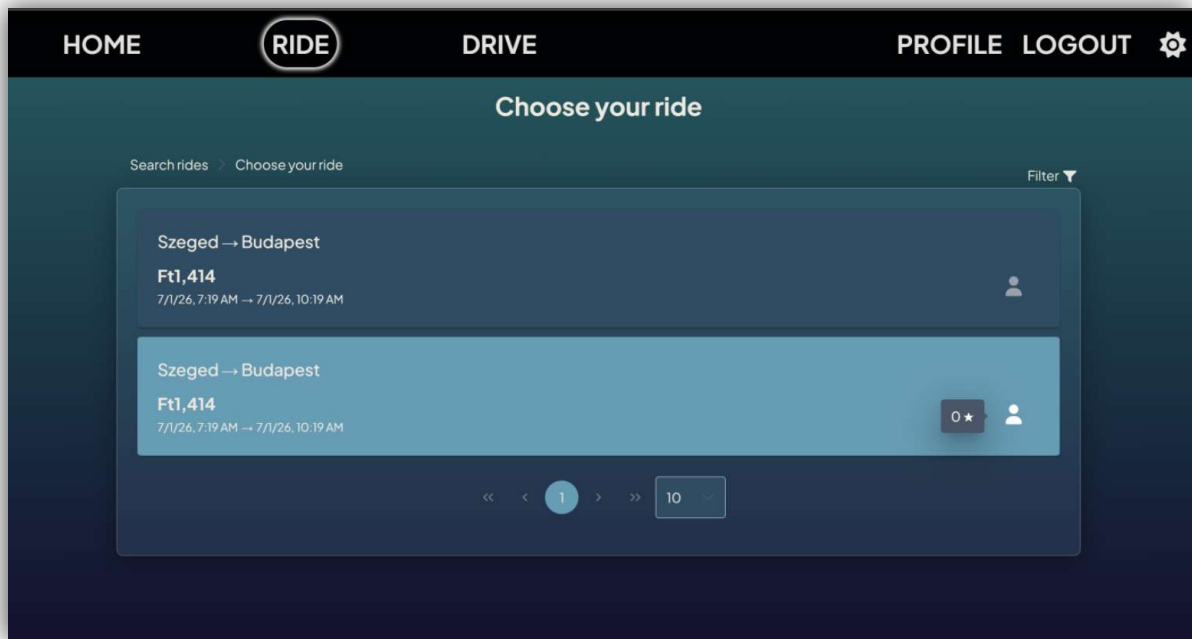
3.1 Sötét/Világos mód

Mai naprakész webalkalmazások egyik **elengedhetetlen design eleme** a sötét és világos mód. Ezeket a sokszor témaként hivatkozott megjelenési módokat az alkalmazás képes automatikusan — az oldalon történő interakció nélkül — felismerni, az operációs rendszer témájával szinkronba hozni.

Azonban fontos megjegyezni, hogy **UX** (felhasználói élmény) szempontjából kötelező a felhasználónak megadni a lehetőséget, hogy ezt az automatikusan inicializált témát megváltoztathassa. Emiatt nem csak háttérben önvezérlően állítódó funkciót alkalmaztam, hanem a felhasználó a **navigációs menüben kiválaszthatja**, hogy sötét vagy világos módban szeretné-e használni az oldalt.



3.1 Ábra, fuvarlista világos módban



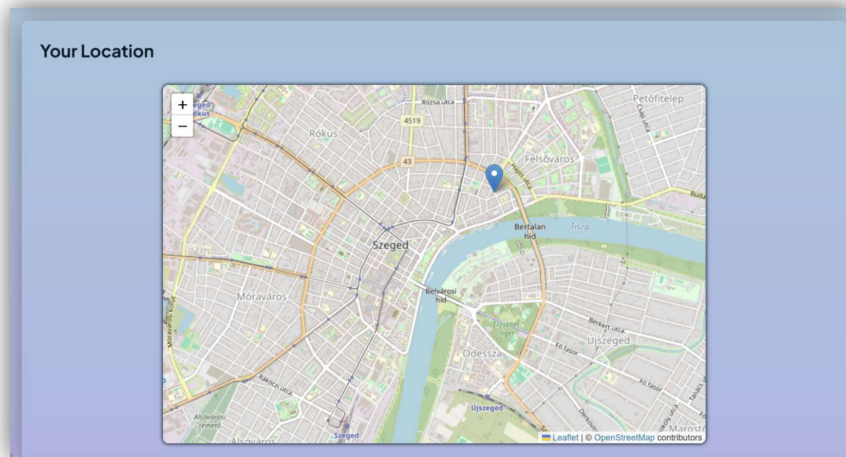
3.2 Ábra, fuvarlista sötét módban

A 3.1-es és 3.2-es ábrán a keresés után szűrt fuvarok vannak. Előbbin az alkalmazás **világos módban** van, így a navigációs menü jobb sarkában egy **hold** szimbólummal ellátott gomb jelzi, hogy rákattintva, sötét módba vált át az alkalmazás. Utóbbi képernyőképen ugyanez az oldal látható **sötét módban**, a navigációs menü gombján itt egy **nap** szimbólum szerepel. Ezek mellett mindkét képernyőképen látszik, hogy a felhasználó szimbólum felé tartva az egeret, megjelenik a sofőr átlag-értékelése.

3.2 Helymeghatározás térképen

Mivel az alkalmazás jelenlegi formájában webes felületként működik, az aktuális pozíció meghatározásához külső térképes alkalmazás megnyitása jelentősen rontaná a felhasználói élményt. Ennek elkerülése érdekében lett része az alkalmazásnak ez a funkció. A térkép képes a **pontos helyszín** meghatározására, ha a kliens eszköz **GPS**-szel van felszerelve. Ha nem érhető el GPS az eszközön, **IP-cím** alapján legalább **város-szintű pontossággal** helyezi el a felhasználót a térképen.

Ez alkalmas olyan esetekre, mint amikor a sofőrrel/utassal megbeszélt **találkozási helyet** szeretnénk megkeresni és **pontosítani**, vagy, bár ritkábban előforduló, de nem elhanyagolható eset az is, amikor nem tudjuk a mellettünk lévő település (esetleg kisebb falu vagy község) nevét.

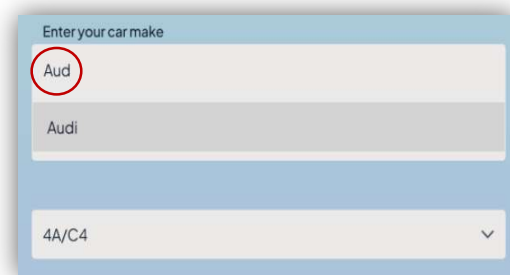
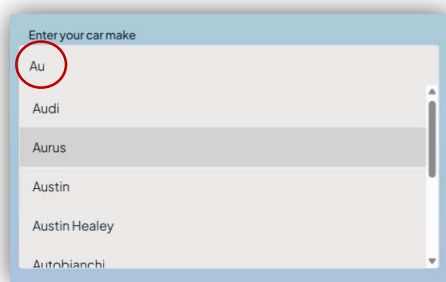


3.3 Ábra, integrált térkép (világos mód)

A **3.3-as ábrán** az integrált térkép látható világos módban. Jól látszik, hogy a használt térkép **interaktív**, nagyítás és kicsinyítés funkció elérhető rajta. A pontos helyszínt egy kék logó jelzi a felhasználók számára.

3.3 Fuvarmegosztás

A fuvarok megosztásához az **úttal** és a szállító **gépjárművel** kapcsolatos információkat kell megosztani a sofőrnek. Ezen az oldalon találkozhatunk többek között dinamikusan frissülő legördülő listákkal, felugró ablakokkal és a megosztás sikerességét jelző toast¹⁴ komponensekkel.



3.4 és 3.5 Ábra, dinamikus márkaszűrő (világos mód)

A **3.4-es** és **3.5-ös ábrán** a dinamikus szűrő látszik folyamatában. Előbbi képen csak az „Au” szöveg került begépelésre, így lényegesen bővebb a találati lista. Utóbbi képen azonban

¹⁴ Toast: Webfejlesztésben a toast kifejezés egy információt hordozó felugró kis ablakot jelent.

már az „Aud” begépett szöveg látható, ami gyakorlatilag az autómárkákat egyetlen egy találatra szűrte le.

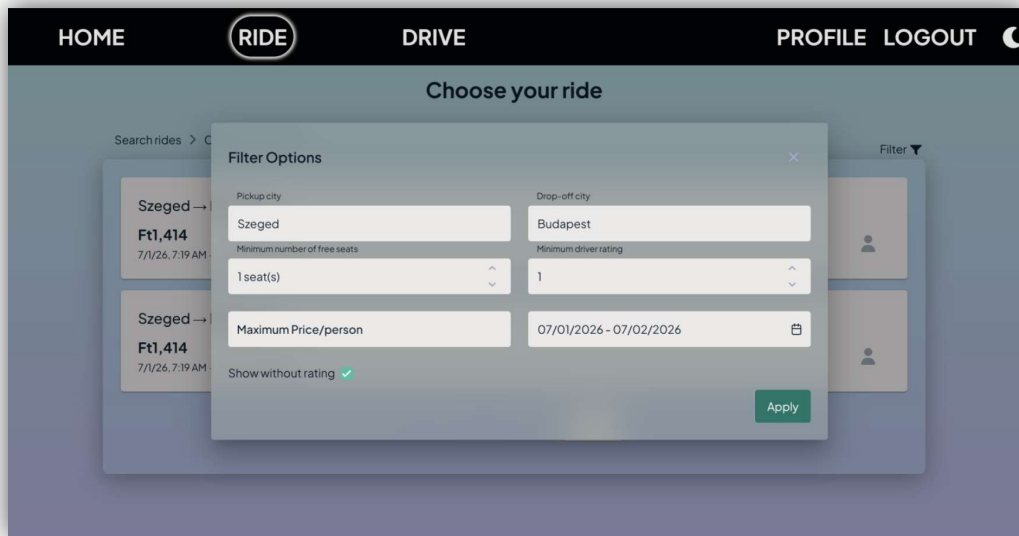
A „fuvar megosztása” gomb megnyomására először kliens oldalon validálásra kerülnek a kitöltött mezők, ha valami nem megfelelő, vagy nincs kitöltve, az egy toast komponenssel jelezve van a felhasználó felé. Sikeres validáció után az alkalmazás egy felugró ablakban kiírja, hogy a paraméterek alapján mennyi az egy utasra jutó ár. Ezt elfogadva megosztásra kerül a fuvar.

3.4 Intelligens fuvar szűrés és lapozás

A felhasználó a meghirdetett fuvarokat kiindulási hely, érkezési hely és dátum intervallum alapján tud **elsődlegesen szűrni**. Ugyanakkor, sok esetben ez túl sok találattal térne vissza, és pontatlan a megfelelő fuvar kiválasztásához. Ezen probléma orvosolása érdekében az alkalmazás lehetőséget kínál az intelligens szűrésre. A keresés optimalizálása és az adatbiztonság érdekében a szűrés teljes egészében szerveroldalon történik. A találatok az alábbi paraméterekkel finomhangolhatók:

- **Szabad helyek száma:** A felhasználó által elvárt minimális ülések száma. Azokat az utakat szűri, amelyekben legalább annyi a szabad hely, mint a szűrőérték.
- **Maximális ár:** Az egy főre jutó költség maximális értéke. A szűrés eredményeként olyan utakat jelenít meg, amelyek díja nem haladja meg a megadott összeget.
- **Minimális értékelés:** A sofőr elvárt legkisebb átlagértékelése a maximálisan adható öt csillagból. Olyan utakat fog kilistázni, ahol a sofőr értékelése legalább akkora értékű, mint a szűrő értéke.
- **Értékelés nélküliek megjelenítése:** Az eddig még nem értékelt sofőrök mutatása. Bekapcsolt állapotban olyan utakat is megjelenít, amelyek sofőrjei még nem kaptak értékelést.

A **3.6-os ábra** mutatja be az intelligens szűrésre alkalmas felületet. Jól láthatók a szűrőparaméterek, és hogy ezek az adat formátuma szerint eltérő beviteli mezőkbe adhatók meg.



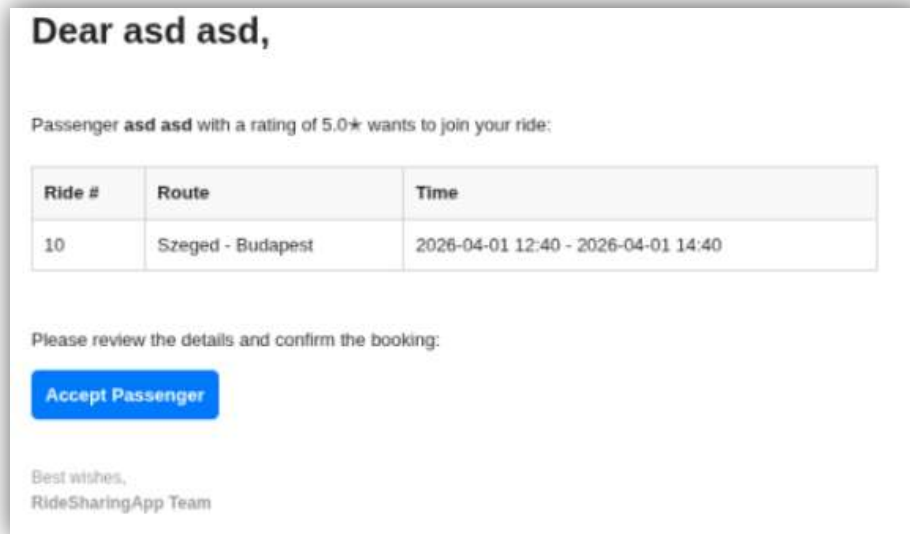
3.6 Ábra, intelligens szűrő (világos mód)

Mivel a szűrt utak száma különösen magas is lehet, a szerver és kliens oldal túlterhelése ellen védekezve, **szerver-oldali lapozást** alkalmaztam. A felhasználó kiválaszthatja, hogy egy oldalon hány találat jelenjen meg, és ez alapján lapozhat a találatok között. A szervertől mindig csak az adott lap találatait kéri le az alkalmazás.

3.5 Utas jóváhagyás

Egy fuvarra történő jelentkezés folyamata a következő: Az utas rászűr az utakra, esetleg használja az intelligens szűrőt a pontosabb találatok érdekében. A kilistázott utak közül az egyikre kattintva, az út részletes adatlapjára jut. Itt leadhatja **csatlakozási kérelmét**, ha a feltételek megfelelnek számára. Fontos, hogy ezzel még nem kerül bejegyzésre az úthoz utasként. A kérelem pillanatában egy **automatikus megerősítő email** érkezik a sofőr címére. Ebben az emailben látja az utas értékelését, valamint egy gombot az utas megerősítéséről. Erre kattintva a sofőr az alkalmazás egy **védett felületére** érkezik, ahol véglegesítheti az **utas jóváhagyását**. A sikeres megerősítést követően bejegyzésre kerül az utas és a szabad helyek száma eggyel csökken.

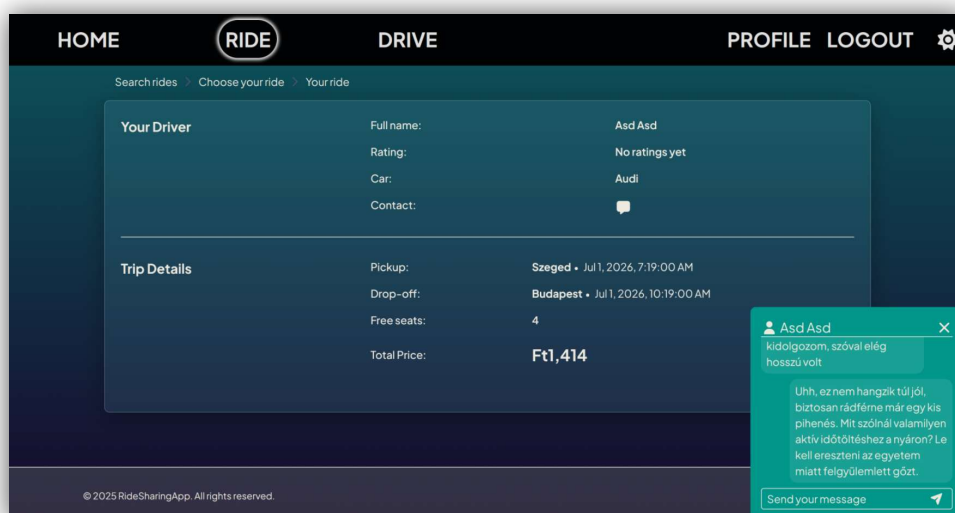
Az **3.7-es ábrán** látható az email, amely a sofőr email-címére érkezik, ha egy utas csatlakozni szeretne az úthoz. Az „**Accept Passenger**” feliratú gomb, mely kék háttérrel és fehér szöveggel rendelkezik, az alkalmazás olyan oldalára vezeti a felhasználót, ahol jóvá lehet hagyni az utast.



3.7 Ábra, jóváhagyó email

3.6 Kommunikáció

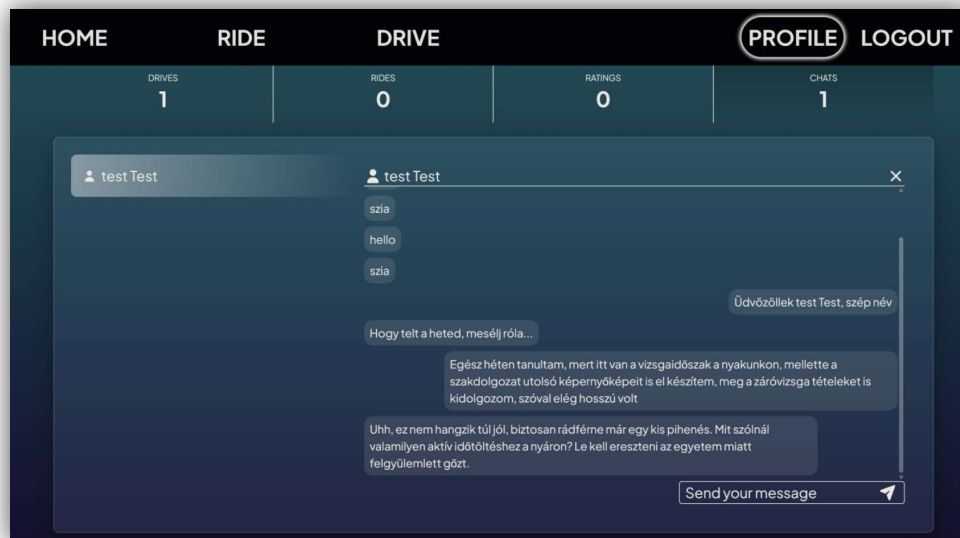
Ahhoz, hogy egy ilyen megbízhatóságon alapuló ügylet — mint a fuvarmegosztás — létre tudjon jönni, elengedhetetlen a felek közötti **valós-idejű kommunikáció**. Az alkalmazásban ennek megvalósításakor is iparági sztenderdeket követve, ezt egy **felúszó füffel** oldottam meg. Először az **utazó** képes **beszélgetést kezdeményezni** a sofőrrel, ezt a sofőr egy útjának adatlapján teheti meg: egy felúszó fül jelzi, hogy kezdeményezzen beszélgetést, ha erre rákattint, kinyílik a chat-ablak. Így még **jelentkezés előtt** egyeztetni tudnak az utazásról, pontosítani tudják a fizetés módját, vagy a találkozási helyszínt.



3.8 Ábra, chat-ablak a kiválasztott út oldalán (sötét mód)

A **3.8-as ábra** jobb sarokban látható a chat-ablak, ami ilyen formátumban csak a kiválasztott út felületén érhető el. A küldött üzenetek, konvenciókat követve, az ablak jobboldalán a kapott üzenetek pedig az ablak baloldalán jelennek meg.

A sofőr **első üzenőként** csak akkor tud írni az utasnak, ha már **jóváhagyta** őt. Ezt a profil menüpont alatt a meghirdetett útjai fülön teheti meg. Már megkezdett beszélgetések bármely felhasználó elér a profilja oldalán.

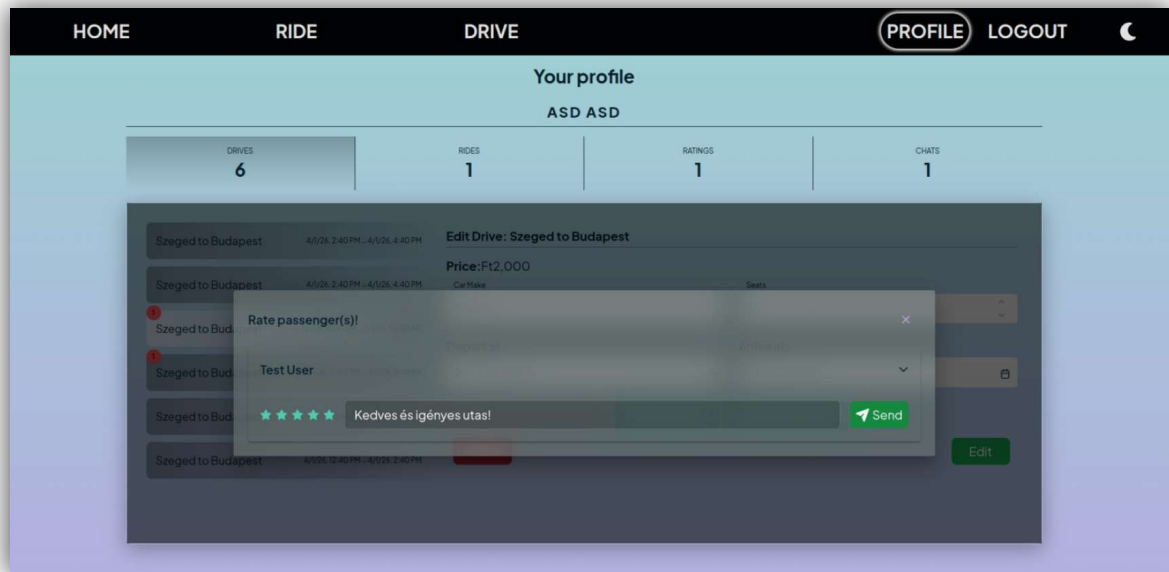


3.9 Ábra, megkezdett beszélgetés a profil fül alatt (sötét mód)

A **3.9-es ábrán** a 3.8-as ábrán már látott beszélgetés van, a profil felületen megnyitva. Itt nem egy kis ablak, hanem egy kinagyított, képernyő méretéhez igazított formában lehet beszélgetni. Az alkalmazás ezen része a már korábban megkezdett beszélgetéseket jeleníti meg, ebből a menüpontból új beszélgetés nem indítható.

3.7 Értékelési rendszer

A kommunikáció sokat segít a bizalom kialakításában a két fél között, de sokszor félrevezető is tud lenni. A **csaló vagy hanyag** felhasználók **ellehetetlenítése** és a **becsületesek kiemelése** érdekében egy értékelési rendszert is megvalósít az oldal. Értékelni csak már megtörtént úthoz tartozó személyeket lehet. A sofőr egy ügyletéhez tartozóan egyszer értékelheti az összes utasát. Ugyanúgy az utas egyszer értékelheti a sofőrjét egy út folyamán. Egy későbbi ügylet alkalmával újra lehet értékelni a már értékelt személyt. Ilyen értelemben úthoz és személyhez is kötött az értékelési rendszer.

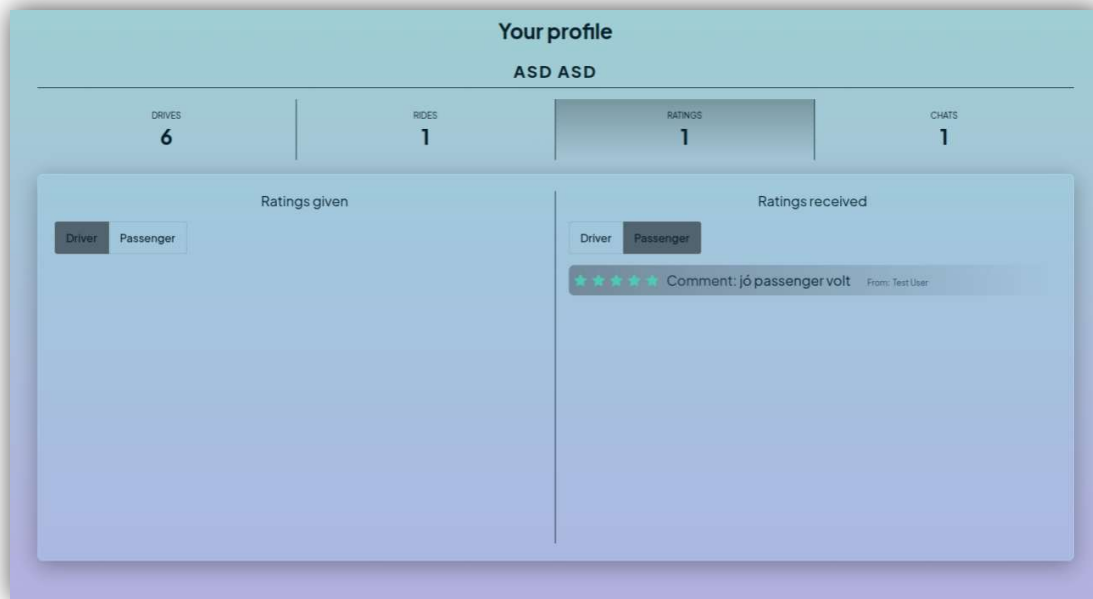


3.10 Ábra, utas értékelése

Ahogy az a **3.10-es ábra** bemutatja, a legmagasabb adható értékelés az **öt csillag**, értékelésnél csak egész csillagokat lehet adni. **Hozzászólást** is **kötelező** a számbeli értékelés mellett, így részleteibe menve lehet leírni a személlyel tapasztaltakat. Fontos megjegyezni, hogy az értékelések elkülönülnek aszerint, hogy sofőrként vagy utasként kaptuk-e őket. Erre azért van szükség, mert nem ugyanazokat az értékeket figyeljük egy sofőrben, mint egy utasban. Például egy sofőr vezethet nagyon biztonságosan, de attól nem biztos, hogy egy becsületesen fizető utas.

A **3.11-es ábrán** azt láthatjuk, hogy értékeléseinket a profil fül alatt tudjuk megnézni. Négy különböző értékelési típus létezik, adott és kapott értékelések sofőrként, illetve adott és kapott értékelések utasként. Ezen a ponton a képernyő két részre van osztva, egyik oldalon a kapott értékelések vannak, másik oldalon az adott értékelések szerepelnek. Mindkét oldalt van egy rádió gomb¹⁵, amelyen kiválasztható, hogy sofőrként vagy utasként néznénk az értékeléseket.

¹⁵ **Rádió gomb:** Olyan gomb, amely meghatározott elemekből egyet enged kiválasztani.



3.11 Ábra, értékelések profil fül alatt (világos mód)

3.8 Értesítési rendszer

Az alkalmazás **kétféle értesítést** használ, egyik formája az **email** címre érkező üzenetek, melyek időzítve és interakcióra is aktiválódnak, másik az applikáción belüli úgynevezett **badge**¹⁶ (kitűző) elemek. Utóbbi egy **passzív** formája a felhasználó figyelmeztetésének; a profil oldalon előző utak fül alatt jelenik meg, egy piros karikában fehérrel írt szám. A szám értéke jelzi, hogy hány embert kell még értékelnie a felhasználónak. Aktív értesítésként emailben értesítjük az utasokat, ha egy tervezett útjuk paraméterei megváltoztak, vagy az utat törölte a sofőr. Ehhez hasonló értesítést kapnak a sofőrök, ha egy utas elhagyja az utat.

Időzített emailt küld az oldal hetente egyszer a sofőröknek és utasoknak is, ha van olyan útjuk, ahol még nem értékelték a másik felet. Egy 2025-ös kutatás alapján az este nyolckor kapott email üzenetek megnyitási aránya elérheti az 59%-ot [11]. Ezért az üzenet **minden héten kedden 8 órakor** megy ki a felhasználók email címére. Az email tartalmaz egy linket, amivel a profiljuk oldalára navigálhatnak, ahol meg tudják nyitni az értékelés modális ablakot, és értékelhetik a sofőrt vagy utast.

¹⁶ **Badge (kitűző):** Kisméretű szimbólum, amely információra hívja fel a felhasználó figyelmét.

4 Főbb funkciók implementálása

Funkciók implementálása során számos bevett gyakorlatot, fejlesztési mintákat használtam. Többek között ilyen a **REST API**, amely behálózza a szerveroldali API végpontok és viselkedések struktúráját. Minden API végpont **JSON** formátumban válaszol, és a frontend is mindenhol ezt várja. Minden hívás a **HTTP** protokollt használja, így ezek főbb metódusai mindegyike fellelhető: **GET**, **POST**, **PUT**, **DELETE**.

4.1 Sötét/Világos mód

A témák választása és szerkesztése pusztán kliens-oldali kód, de a legjobb gyakorlatok és minták használata itt is megjelenik. Ehhez a funkcióhoz **Angular Service**-t használtam, ez egy olyan példányosított osztály, mely életciklusát az Angular kezeli. Kifejezetten olyan feladatokra használják, ha valamilyen funkcionalitást, vagy adatot több komponensen átívelően kell megosztani. A témák állításáért a *ThemeService* felel, létrejöttékor a **matchMedia API** által lekérdezi a felhasználó operációs rendszer szintű téma preferenciáját, és ezzel inicializálódik.

A frontend oldal színeit előre definiált SCSS és CSS változók határozzák meg. SCSS változókban minden színnek van egy *light* és egy *dark* verziója, míg CSS-ben csak egy központi változó van. Ennek a központi változónak a színe alapból az SCSS *light* verzióját kapja, azonban, ha **HTML body**-hoz hozzá van fűzve a *dark-theme* osztály, minden változó átvált a sötét varianciájára. A korábban említett *ThemeService* feladata nem több, mint egy a navigációs menüben elhelyezett gomb hatására hozzáadni, vagy levenni a body-hoz fűzött *dark-theme* osztályt, ezzel átállítva az alkalmazás színeit.

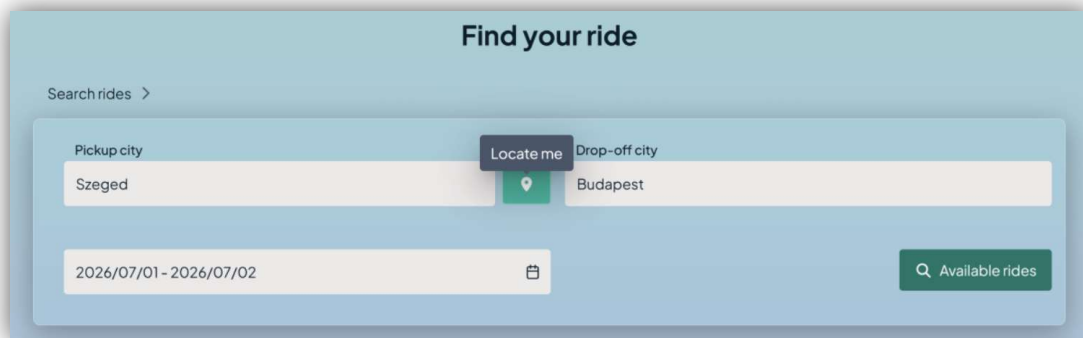
4.2 Helymeghatározás térképen

Az **interaktív térkép** megjelenítéséhez a **Leaflet.js** nyílt forráskódú JavaScript könyvtárat használtam. A *map-component* jeleníti meg a térképet, ennek HTML template fájlja eleinte üres, magát a térképet dinamikusan tölti be a komponens TypeScript fájlja. Ebben fájlban található egy **@Input** dekorátorral ellátott **GeoLocation objektum**, ez kötelezi a szülő komponensre, hogy átadja az értéket a *map-component*-nek. Az input objektum változását az Angular egy **lifecycle hook**¹⁷ metódusa — az *ngOnChanges* — figyeli, és a változásoknak megfelelően helyezi el a térképen elhelyezett **jelölő szimbólumot**, ami a felhasználó aktuális

¹⁷ Lifecycle-hook: Azok az események, amelyek egy komponens létrejötte és megsemmisülése között történhetnek.

pozícióját jelzi. Jelenleg a komponenst csak az **utazás kereső** felület használja, de architektúrája miatt könnyen integrálható más komponensek részeként is.

Utak keresésénél kitöltendő kiindulópont mezője mellet található egy „Locate me” gomb, amely hatására a **Geolocation API** lekéri az eszköz pozíciójának koordinátáit. A térképen a koordináták pozíciójára navigál, és elhelyezi a jelölő szimbólumot. Emellett egy **HTTP GET kérést** küld a backend felé a koordinátákkal paraméterezve, mindezt a *GeocodingService* segítségével. A szerver a kérés hatására meghívja az **OpenWeatherMap API** „reverse geocoding” végpontját, amely a koordináták alapján visszaadja a város nevét. Ezt adja válaszként a frontendnek, és kitölti a kiinduló pont mezőt az értékkel. Ezt a funkciót a **4.1-es ábrán** lehet látni, ahol tooltip¹⁸ felirat részeként jelenik meg a „Locate me” szöveg a gomb felett.



4.1 Ábra, a „Locate me” gomb

4.3 Fuvarmegosztás

Az autó pontos paramétereinek kitöltésénél dinamikusan frissítjük a listákat. Először az autómárkát kell kiválasztani: ennek a listájában egy beviteli mező segítségével szűrhetünk. A *drive-component* TypeScript fájlja figyeli a beviteli mezőt, és az **RxJS *debounceTime()*** metódusának köszönhetően **0,3 másodperces késleltetéssel** értesíti az őt figyelő komponenst. Azonban, ha a 0,3 másodpercen belül újabb változás van a mezőn, akkor **eldobja az előzőleg feljegyzett értesítést**, és újra kezdi a 0,3 másodperces várakozást. A várakozási idő lejártával — vagyis az értesítés elkapásával — lekéri a szervertől a beírt karakterekre illesztett autómárkákat. Ez a késleltetés elengedhetetlen eleme egy optimálisan működő, nagyobb forgalommal dolgozó keresőnek, mert enélkül — a karakterenként elküldött kérésektől — gyorsan túlterhelődne a backend.

¹⁸ **Tooltip:** Rövid információs komponens, mely általában egér fölé tartásra aktiválódik.

A márka kiválasztása azonnali kérést indít a szerver irányába, a modell adatokért, amiket először **megkísérel** az adatbázisból **gyorsítótárazni**. Ha ez sikertelen, akkor egy **külső szolgáltatástól** kérjük le az adatokat, és mentjük le őket. Ennek a folyamatnak a részletes technikai megvalósítását **6.2 alfejezet** mutatja be. Ugyanez a folyamat megy végig a további mezőkön, amíg a pontos motortípust is megismeri a szerver.

A gépjármű adatai mellett, az úttal kapcsolatos adatok is kötelezőek, validálva vannak a kliens oldalon: például nem adható meg a mostaninál régebbi dátum. Ha minden mezőt megfelelően kitöltöttünk, a **„fuvar megosztása” gombra** kattintva az alkalmazás elindítja az **árkalkulálást**, ez akár 1-2 másodpercet is igénybe vehet. Ez idő alatt a felhasználói felület felső részén egy folyamatjelző sáv jelzi, hogy hamarosan válaszol az oldal. Ha az egy főre jutó ár kiszámításra került, és ezt a sofőr elfogadta, az alkalmazás perzisztensen menti az adatokat.

4.4 Intelligens fuvar szűrés és lapozás

Az intelligens szűrő — az alapértelmezett szűrővel ellentétben — **HTTP POST** kérést küld a szerver felé, mert lényegesen több paraméterrel dolgozik. A POST kérés törzsébe kerülnek a szűrő paraméterek, amikkel a szerver dolgozik tovább.

A Spring Bootnak köszönhetően, könnyedén **@RequestBody** annotációval deszerializálható az egyébként JSON formátumban érkező törzs. Szerveren ezt egy *RideFilterRequest* nevű **Java record típusként** kezeljük. Ahogy azt korábban említettem (lásd: 2.3 alfejezet) a **@Service** annotációval ellátott osztályok felelősek az üzleti logikáért. Ezek felelnek a validációért, hívják meg az adatbázis réteget és egyéb más függőségeket. Ahhoz, hogy komplexebb SQL lekérdezéseket végezzünk nem elegendő a már ismertetett *JpaRepository*, ezért a fuvarokat manipuláló/lekérő *Repository* interfész kibővül a **JpaSpecificationExecutor** interfésszel. Ez képes predikátumokat tartalmazó *Specification* objektumokat fogadni, és ezekből egy komplex SQL lekérdezést összeállítani.

A **Specification** objektumok legyártásáért egy dedikált, **tovább nem örököltethető** (final kulcsszóval ellátott) osztály felel. Ez a *JourneySpecificationFactory*, mely **statikus metódusokkal** *Specification* objektumokat hoz létre. Ezek érték nélküli szűrőparaméterek esetében *null* értékkel térnek vissza, így a szűrő figyelmen kívül hagyja őket. A statikus metódusokhoz nem kell példányosítani az osztályt, így nem használunk felesleges memóriát. Az osztály egy privát konstruktorral rendelkezik, így nincs is lehetőség objektumot létrehozni belőle.

A Specification legyártására egy szemléletes példa az utak szűrése a sofőrjeik értékelése alapján:

1. **Allekérdezés készítése:** A rendszer készít egy allekérdezést az értékelésekből, melyben az értékelések átlagával aggregációt végez.
2. **Adatok összekapcsolása:** Az allekérdezésben összekapcsolja az értékelés táblában tárolt értékelt személy oszlopát, a fuvar táblában lévő sofőr oszlopával. Az összekapcsolás mellett egy „és” feltételben megszabja, hogy az értékelés iránya utastól sofőrhez legyen, erre a RatingType Java enum típust használja.
3. **Nem értékelt sofőrök kezelése:** Nem értékelt sofőrök esetében az értékelésük átlaga *NULL* érték lenne, ezt az *SQL COALESCE* függvénnyel 0.0-ra állítja a rendszer, így további számításokat lehet végezni az eredményen.
4. **Végső kiértékelés:** A kapott átlagot összehasonlítja a szűrőben megadott minimum értékkel, és csak az ennél nagyobb átlagértékeléssel rendelkező sorokat adja vissza. Amennyiben a szűrőben engedélyezte az értékeléssel nem rendelkező sofőrök szűrését, a 0.0-ra állított értékelésüket is visszaadja.

```
public static Specification<Journey> findByRating(double rating,
boolean showWithoutRating) {

    return (root, query, cb) -> {
        // create subquery
        Subquery<Double> subquery = query.subquery(Double.class);
        Root<Rating> ratingRoot = subquery.from(Rating.class);

        // select average rating
        subquery.select(cb.avg(ratingRoot.get(Rating_.VALUE)));

        // join Rating table's rated col to Journey table's driver
        col
        subquery.where(cb.and(
            cb.equal(ratingRoot.get(Rating_.RATED),
            root.get(Journey_.DRIVER)),
            cb.equal(ratingRoot.get(Rating_.TYPE),
            RatingType.PASSENGER_TO_DRIVER)
        ));

        // if rating was null use 0.0
        if (showWithoutRating) {
            return cb.or(
                cb.greaterThanOrEqualTo(cb.coalesce(subquery,
                0.0), rating),
                cb.equal(cb.coalesce(subquery, 0.0), 0.0)
            );
        }
        return cb.greaterThanOrEqualTo(cb.coalesce(subquery, 0.0),
```

4.2 Kódrészlet, Specification objektum értékelés alapján

A *sofőr* értékelése alapján készült *Specification* objektum elkészítését a **4.2-es kódrészlet** mutatja be. Az objektumot egy lambda kifejezés hozza létre. Ez azért működik, mert a *Specification* objektum egy funkcionális interfész, vagyis egyetlen absztrakt metódusa van, a *toPredicate()*. Ez három objektumot vár:

- *root* (***Root<T>***): Ez reprezentálja magát az adatbázis entitás osztályát. (*FROM*)
- *query* (***CriteriaQuery<?>***): Ezzel lehet módosítani a lekérdezés kimenetét. (*GROUP BY, ORDER BY*)
- *cb* (***CriteriaBuilder***): Ez készíti a lekérdezés feltételeit, operátorait. (*AND, OR, AVG*)

A lekérdezés összeállítása során egy SQL tábla oszlopaira a *Root<T>* példányok *get()* metódusával hivatkozhatunk. Ennek paraméterként az oszlopot reprezentáló **Java entitás adattagját** kell átadni. Jó gyakorlat, ha nem egy kódba beégetett stringet kap értékként, mert később változhat az osztály neve. Ezért használtam a **Hibernate JPA Modelgen** függőséget, mely dinamikusan tárolja, frissíti az osztály adattagjainak neveit.

A *Specification* objektumokat egymáshoz „és” **feltétellel** fűzve, kapunk egy összetett objektumot, ami tartalmazza a szűrést. Emellett készítünk egy **Pageable példányt**, ami a lap méretét, lapra jutó rekordok számát és rendezést tartalmaz. Ha mindkét készített elemet átadjuk a megfelelő *Repository findAll()* — minden értéket visszaadó — metódusának, akkor egy **Page objektumot kapunk** vissza, ami tartalmazza a szűrt, rendezett adatokat, a lapozás paramétereivel limitálva. Ezt az objektumot küldi el a szerver a frontendnek, és az ebben tárolt adatokat jeleníti meg az oldal.

4.5 Utas jóváhagyás

Az utazás és a felhasználók között **több a többhöz (N:M) kapcsolatot** kell létrehoznunk, hiszen egy utas többször is utazhat, és egy utazáshoz több utas is tartozhat. Ezt a Spring Data JPA a **@ManyToMany** annotációval könnyedén lekezelem, létrehoz egy dedikált táblát, ahol az összekötött táblák kulcsait tárolja idegen kulcsként. Ahhoz, hogy az utast jóvá kelljen hagynia a sofőrnek, szükségünk van **extra oszlopokra** ezen a táblán. Mivel a JPA a háttérben generálja a kapcsolótáblát, ehhez nem lehet hozzáférni, nem lehet kiegészíteni további oszlopokkal. Ennek a megoldására **explicit** entitásként hoztam létre a **kapcsolótáblát**, ami az **N:M kapcsolatot kettő darab 1:N kapcsolatra bontja**. Két adattaggal bővül ez az entitás az idegen kulcsok mellett, egy *String* típusú *acceptToken* nevű kulccsal és egy *accepted* bináris értékkel.

Utóbbi eleinte mindig hamis értéket kap, ezzel jelezve, hogy az utas még nincs elfogadva. A tábla kulcsát a **Java UUID API** generálja az utas csatlakozási kérelme pillanatában. Ugyanebben az időben készítünk egy a kulcsot paraméterként tartalmazó URL-t, amit a sofőr e-mailben kap meg.

A sofőrnek tehát **email** érkezik az **utas utazási kérelméről**, az email tartalmazza az **utas értékelését**, és egy gombot, ami a Ride Sharing App jóváhagyási oldalára navigálja őt. Olyan link van a gombba ágyazva, amely paraméterként **tartalmazza** az utas kérésének **UUID tokenjét**. Ezt a **frontend kinyeri** az URL-ből és ha a jóváhagyási oldalon lévő „Elfogadás” gombra kattint a sofőr, egy **POST kérést** irányít a szerverhez, amely **igazra állítja** a rekord **accepted oszlopát**, és **lecsökkenti** az úthoz tartozó **szabad helyek** számát eggyel.

4.6 Kommunikáció

A beszélgetéshez elengedhetetlen chat-ablakot a *chat-component* újra felhasználható komponens valósítja meg. Ez a komponens felel a **Websocket felállításáért**, a beszélgetés megjelenítéséért és az üzenet küldéséért. Tekintettel arra, hogy nem csak egy oldalon van alkalmazva, számos testreszabhatóságot elősegítő tulajdonsággal rendelkezik. A szülő-gyerek komponensek közötti kommunikációért modern Signal¹⁹ típusokat használ. A szülő komponenstől **érkező adatok fogadását** és követését az **InputSignal**-ok, míg a gyerek komponens által küldött **adatok küldéséért** az **output() API** felel. Emellett **ModelSignal** típust is alkalmaz, mely különlegessége, hogy output-ként is képes viselkedni, így ugyanazt az adatot egyszerre figyelheti/manipulálhatja a szülő és gyerek komponens is.

Ahogy a 3.6-os alfejezetben részleteztem, először egy kiválasztott út adatlapján lehet találkozni a komponenssel. A *chat-component* a szülő komponens után kettő másodperccel töltődik be, először egy felnyitható fül formájában. Ez a szülő komponensben egy beúszó **CSS animáció** hatására jelenik meg. Ha a felhasználó rákattint a felnyitható fülre, egy **boolean** értéket körülölelő **ModelSignal** értéke igaz lesz, és ezzel kinyílik a chat-ablak. Ez az érték azért van kétirányúan megosztva, mert fontos, hogy a szülő konténer is tudja, ha bezárjuk a chat ablakot. Ez a Signal figyelve van az *effect API* segítségével, ha igaz értéket kapott, tehát szétnyitottuk a chat-ablakot, akkor elindul a websocket inicializálása és korábbi üzenetek betöltése.

¹⁹ **Signal**: Angular reaktív típus, amely egy értéket csomagol be, és értesíti rá feliratkozó elemeket az érték változásáról.

A chattel kapcsolatos kérésekért a *ChatService* Angular Service felel. Ez indítja el a websocket felett álló **STOMP klienst**, és iratkozik fel a megfelelő topicra²⁰. A STOMP elindításához meghívjuk a hoszt url */ws* végpontját, amely egy tartós websocket kapcsolatot hoz létre [9]. Backend oldalon a *WebsocketConfig* konfigurációs osztály felel a websocket szerver létrehozásáért. Itt van definiálva, hogy a */ws végpont* meghívására jöjjön létre a kapcsolat.

A *ChatSimpController* felelős az üzenetek koordinálásáért, a frontend üzenetküldésnél egy olyan JSON formátumú objektumot küld, amiben szerepel a küldő, fogadó felhasználóneve és az üzenet szövege.

Ezt az üzenetet egy Java objektummá deszerializálja a Jackson függőséggel, illetve a szerver időbélyeggel egészíti ki, hogy konzisztensen meg lehessen később jeleníteni az elmentett üzeneteket.

A kontroller az üzenetet a *SimpMessagingTemplate* osztály segítségével egy olyan csatornára küldi, amelyen két felhasználó osztozik [10]. Ezt a csatornát a felhasználónevek alapján van meghatározva (*/topic/private-messages/{username1-username2}*). Ehhez segítségül van egy normalizáló metódus, mely determinisztikusan, ABC-sorrend szerint fűzi össze a két nevet.

4.7 Értékelési rendszer

Az értékelésekkel kapcsolatos figyelmeztetések a 3.8 alfejezetben részletezve jutnak el a felhasználókhoz. A felhasználók értékelésüket egy öt csillagból álló csúszkán adhatják meg. A csúszka a **PrimeNG csomag** része.

A sofőr akár több értékelést is adhat egy utazáshoz köthetően, hisz több utasa is lehet. Emiatt egy modális ablakban vannak felsorolva az értékelendő személyek. A személyek nevei úgynevezett harmonika — angol nevén **accordion** — szétnyitható elemekben jelennek meg. Ezt lenyitva jelenik meg a komment szekció, az értékelő csúszka és az értékelés elküldésére alkalmas gomb. Küldésre a frontend ellenőrzi, hogy a komment szekció ki lett-e töltve, ha nem, akkor toast komponenssel jelzi hiányát.

Az értékelés elküldésére két végpont van: */api/rating/add/as-passenger/{id}* és */api/rating/add/as-driver/{id}*, előbbi a sofőrök, utóbbi pedig az utasok értékelésére szolgál. Az *id* változó az utazás adatbázis táblában tárolt elsődleges kulcsa. Mindkét végpont HTTP POST

²⁰ **Topic:** STOMP világában egy topic az a végpont, ahova az üzenetet küldjük.

metódust használ, a törzsben egy olyan JSON objektum van, amely tartalmazza az értékelés értékét, a kommentet, az értékelő és értékelt felhasználónevét. Kiemelendő, hogy a szerveroldalon JWT tokenből nyerjük ki a hívó felhasználónevét, így nem hamisítható más értékelése.

Szerveroldali architektúra, ahogy azt a 2.3-as alfejezetben említettem, a **Spring konvencióit** követi, tehát a *RatingController*, az értékelésekért felelős *@RestController* a *RatingService*-ben leírt üzleti logikát hívja meg. Itt is, tehát szerveroldalon is, **validálva** van a **komment** üressége, hisz egy API nem csak a frontendről hívható meg, hanem bármely más API kliensről. Továbbá ellenőrzi, hogy az utazás már **elkezdődött-e**, valóban az illetőhöz köthető az utazás és hogy van-e **korábbi értékelése** ehhez az úthoz az értékelt személy irányába. Ha az ellenőrzések bármelyikének nem felel meg, egy **dedikált kivétel dobódik**, amelyben üzenetként a hiányosság leírása szerepel. Ennek a folyamatnak a részletes kidolgozása az 5. fejezetben ismerhető meg.

Értékelést nem csak adni lehet, fontos, hogy utólag meg is tudjuk nézni őket. A frontend két részre osztása **CSS Grid**-del lett megoldva, amelyet két oszloppal definiáltam. A megjelenítésnél látható, hogy kitől kaptuk az értékelést, mit írt hozzászólásként és hány csillagot adott. Itt is a PrimeNG komponensét használom a csillagok megjelenítésére, viszont **disabled**-re van állítva a HTML kódban, ezzel jelezve, hogy ez egy view-only állapot. Fontos, hogy a **reszponzivitás** megőrzése érdekében a **hozzászólás le van vágva**, és három darab pont jelzi, ha folytatódik. A teljes hozzászólást egy **tooltip**-ben láthatjuk, ha az egerünket a levágott hozzászólás felett tartjuk. A dolgozatban szem előtt van tartva, hogy mobilos felhasználóknak is maximális élményt nyújtssa az alkalmazás. Azonban, mobilon nincs egér, amit felette tarthatna a felhasználó, ezért egy kis **információs logó** látható a **felső indexben**. Erre rákattintva megnyílik egy **popover**²¹, amiben szintén a hozzászólás szerepel a tooltiphez hasonló módon, teljes egészében.

Szerveroldalon is ugyanúgy szét választva lehet lekérni az értékeléseket, így **négy különálló SQL lekérdezés** van rá megírva. A Rating entitásban használt egyedi Java Enum miatt nem lehet az egyszerű Spring Data JPA metódusból származó lekérdezés funkcióját alkalmazni. Helyette a **JPQL** használata a kézenfekvő, amely SQL-hez hasonló formátumban

²¹ **Popover**: Olyan elem, amelyet, ha meghívunk, a tartalom fölé kerül, általában mellé kattintással bezárható.

íródó, de Javában megírt entitásokra és típusokra hivatkozó szintaxis. Ezt a `@Query` annotációval tehetjük meg, a lekérdezést a *value* változó értékének kell adni stringként.

Eleinte nem tölti be a szerver az összes adatot, hanem egy gomb hatására **ötösével töltődnek** be. Ennek érdekében a válaszadás itt is Page objektumban van, mert szükségünk van az összes rekord számosságára, hogy a gomb ne jelenhessen meg, ha már nincs több adat. Ahhoz viszont, hogy JPQL-ben megírt lekérdezés tartalmazzon ilyen információt, hozzá kell adni bónuszként a számosság lekérdezését. Ahogy az a **4.3-as kódrészletben** is látszik, ezt a bónusz lekérdezést a `@Query` annotáción belül a *countQuery* változónak kell átadni értékként.

```
@Query(
    value = "SELECT r FROM Rating r WHERE r.rated = :driver " +
            "AND r.type = " +
            hu.ridesharing.entity.RatingType.PASSENGER_TO_DRIVER + " ORDER BY r.value",
    countQuery = "SELECT count(r) FROM Rating r WHERE r.rated = " +
            ":driver " +
            "AND r.type = " +
            hu.ridesharing.entity.RatingType.PASSENGER_TO_DRIVER + " "
)
Page<Rating> findByDriverRated(User driver, Pageable pageable);
```

4.3 Kódrészlet, adott értékelések lekérdezése sofőrként

4.8 Értesítési rendszer

Az értesítések passzív formája a profil oldal alatt a „Rides” vagy „Drives” fül alatt jelenhet meg. Ez letisztult formája a felhasználó értesítésének, de implementálása nem evidens. A megjelenítéshez a **PrimeNG Badge** komponensét használtam, amelynek meg lehet adni egy számértéket, amit megjelenít. Elhelyezése a sor bal felső sarkába, abszolút CSS pozicionálással van megoldva.

A „Rides” és „Drives” fül alatti felsorolt adatok, vagyis az utazások, ahol a felhasználó vagy utasként, vagy sofőrként vett részt, olyan model és DTO kommunikációs osztályban vannak, mint amiket utazásokra keresésnél használtunk. Ezekben az osztályokban azonban nem szerepel, hogy hány embert kell még értékelni. Ezért ebből az osztályból származtatva létezik egy másik kommunikációs osztály, amely kiegészül ezzel az információval (nem értékelt utasok számával), és ez van alkalmazva.

A még értékelendő utasok számának meghatározása az adatbázis séma összetettsége miatt több lépésben végzendő:

1. **Utazások lekérése:** Lekéri azon utazásokat, ahol a felhasználó a sofőr.

2. **Jóváhagyott utasok lekérése:** Lekéri minden utazáshoz a jóváhagyott utasokat.
3. **Értékelt utasok szűrése:** Leszűri a már értékelt utasokat.
4. **DTO objektummá alakítás:** Átalakítja kommunikációra alkalmas, értékelendő utasokkal kiegészített DTO objektummá.

A szerver leterhelésének elkerülése végett a DTO objektumok válaszadása a „Ride” és „Drive” fül alatt is a már korábban ismerttetett lapozási technikát használják. Itt is Page objektumba vannak csomagolva a DTO példányok, így a felhasználó ötösével tudja betölteni az adatokat.

Aktív formájú értesítések legtöbb esetben automatikusan, egy **művelet végrehajtása** után email formában kerülnek kiküldésre. Ilyenre példa amikor töröljük, vagy elhagyjuk az utazást, a másik fél emailben kap értesítést erről.

Értékeléshez köthető aktív értesítés pedig a 3.8 alfejezetben említett **időzített email**. Ezek az emailek összesítve figyelmeztetik a felhasználót az értékelésekről. A már említett `@Query` annotáció segítségével megírt egyedi JPQL gyűjti össze az emailre alkalmas felhasználókat. A lekérdezés lényege, hogy **kiszűrje** azokat az utazásokat, ahol már **minden értékelendő fél értékelve lett** egy bizonyos felhasználó által.

Mivel sok felhasználó esetén ez lényegesen sok email kiküldését is jelentheti, a *dev.failsafe* csomag **Failsafe és RetryPolicy** implementációja van használva. A RetryPolicy figyel a „**Too Many Request**” HTTP hibákat, amelyek akkor jönnek válaszként, ha túllépjük a szolgáltató limitjét. Ebben az esetben 10 percet vár és újra próbálja. Ezt a folyamatot maximum ötször viszi végbe.

Az időzítéshez a Spring Boot `@Scheduled` annotációja van egy metódus felett alkalmazva. Ennek CRON kifejezéssel van beállítva, hogy mikor és milyen gyakran fusson le az időzített metódus.

5 Globális hiba kezelés

Az alkalmazás több pontján is **egyedi kivétel osztályok** vannak használva. Ezek a futás során a fejlesztőknek nagyon fontos információk, mert pontosan jelzik, hogy milyen jellegű hiba történt. Ha azonban ezeket nem kezelnénk le, a Spring Boot keretrendszer futását megszakítva alapértelmezetten egy „500 Internal Server Error” hibával térnénk vissza a kliens felé.

A REST API és a HTTP protokoll szabványai viszont megkövetelik, hogy a hibák jellegét a megfelelő, beszédes **HTTP státuszkódokkal** kommunikáljuk. Emiatt hoztam létre a *GlobalExceptionHandler* osztályt, amely egy központosított hibakezelő. Ez a komponens globális szinten elkapja a dobott kivételeket, és tartalmukat szabványos HTTP válaszokká alakítja, melyben a megjelenő üzenet már a frontenden megjeleníthető formátumban van.

Az egyéni kivétel osztályok és hozzájuk rendelt HTTP státuszkódok a következők:

- **Erőforrás nem található (404 Not Found):** Ez a kód pontosan jelzi a kliens irányába, hogy a szerveren a kért erőforrás nem elérhető.
 - **DriverNotFoundException:** Ez az egyedi kivétel akkor dobódik, ha egy adott azonosítóval rendelkező sofőr nem szerepel az adatbázisban, de a kliens megpróbálta elérni.
 - **DriveNotFoundException:** Akkor fordul elő, ha egy utazást nem talál a szerver.
- **Jogosultsági probléma (403 Forbidden):** Ez a kód azt jelzi, ha egy felhasználó sikeresen autentikálta magát, de a művelethez nincs joga.
 - **ForbiddenException:** Akkor használjuk, ha egy olyan végpontot próbál elérni a felhasználó, ami nem hozzá tartozik. (Például másik személy chat-előzményeit)
- **Hibás kliens kérés (400 Bad Request):** A 400-as kód a kliens kérésének hibáját jelzi, nem megfelelő a formátum.
 - **BadRequestException:** Általános validációs hiba esetén dobódik. (Például nem adott hozzászólást az értékeléshez.)
- **Szerveroldali belső hibák (500 Internal Server Error):** A kliens kérése megfelelő volt, de szerveroldalon valamilyen váratlan hiba történt a végrehajtás során.
 - **EmailSendingError:** Akkor fordul elő, ha az email küldés megghiúsul, mert például a szolgáltató nem elérhető.
 - **ChatException:** Az üzenetküldő modulon történő belső hibákat jelzi.

- **JoinRideException:** Ha csatlakozás folyamatában egy rendszerhiba, vagy adatbázis inkonzisztencia lépett fel.
- **RatingException:** Értékelések mentésekor, vagy összeszámolásakor felmerülő belső logikai hiba esetén dobódhat.

6 Út árának számítása — PriceCalculatorService

A rendszer egyedisége, az **egy utasra jutó költségek kiszámítása**. Ahhoz, hogy ezt meg tudjuk tenni több egymástól függő és független adatra van szükségünk, mint ülések száma, autó évjárat, tervezett út hossza, amortizáció értéke kilométerenként, üzemanyag-fogyasztás és az üzemanyag ára. Az alkalmazás az imént említett paraméterek kiszámítására különféle megoldásokat alkalmaz, hogy végeredményként egy objektív ár-számítást kapjunk.

Az architektúra hatékonysága és a **clean code** elvek betartása végett a számítás egy a Spring keretrendszer által menedzselte Service osztályra van bízva, a *PriceCalculatorService* - re. A Service *getPrice* metódusa — amit a *DriveController* hív meg — hét kritikus paraméter alapján határozza meg a végleges díjat. A függvény meghívása előtt a Controller lekérdezi a hosszúsági és szélességi koordinátáit az indulási és az érkezési városoknak a Geocoding API segítségével. Az ár-kalkuláló osztálynak függőségeiként felsorolhatók a *RouteService* és *RouteRepository*, melyek a megtett út hosszát hivatottak visszaadni, illetve a *RapidApiCarService*, ami segítségünkre van az évjárat, fogyasztás és férőhelyek számának elérésében. Mindkét felsorolt Service **gyorsítótárazási mechanizmussal** van ellátva, mert a külső API, amit használnak véges számú ingyenes kéréssel használhatók.

6.1 Tervezett út hossza

A *RouteService* osztály felel az **út hosszának kiszámításáért**, az Open Route Service külső API bevonásával. A kommunikáció itt is HTTP protokollon keresztül, az adatcsere JSON formátumban zajlik.

Tartozik az osztályhoz kettő darab kommunikációhoz használt belső osztály: *ORSRequest* és *ORSResponse*. Előbbit a kérés törzsébe küldi el, ennek a belső osztálynak két adattagja van: egy két dimenziójú lebegőpontos tömb (locations) és egy string lista (metrics). A „locations” a kiindulási és érkezési pontok koordinátáit hordozza, a „metrics” pedig a válaszban várt értékeket. Például, ha a „metrics” -ben benne van a distance szó, akkor a válasz tartalmazni fogja a távolságot méterben. Utóbbi, vagyis az *ORSResponse* belső osztály, kettő két-dimenziós lebegőpontos tömböt zár magába (distances és durations).

Az osztálynak egy *getResponse* metódusa van, ami meghívja a koordinátákkal az API -t és visszatér egy *ORSResponse* objektummal. Ennek az objektumnak a distances adattagjából szerzi meg az ár-kalkuláló osztály a távolságot méterben. Ezt a számadatot a későbbi számítások érdekében kilométerre váltjuk át.

A külső **API hívások minimalizálása** és a válaszidő csökkentése érdekében a rendszer saját **gyorsítótárazási** mechanizmust végez, tehát az útvonalak hossza két város között egynél többször nincs lekérve. A *Route* Java osztály reprezentálja az eltárolandó entitást, ami eltárolja a két végpont nevét, koordinátáit, a köztük lévő távolságot és az út megtételére átlagosan töltött időt. Az entitás egyedi azonosítója egy összetett kulcs, a *RouteId*, ez a két város nevéből van összefűzve. Ehhez van egy speciális normalizáló metódus, hogy ne tároljunk redundáns adatokat. Az algoritmus mindig a kiindulási pontnak választja az ABC-sorrendben hamarabb következő várost, így a „Szeged — Budapest” és a „Budapest — Szeged” utak csak egyszer lesznek letárolva, mert a távolság ugyanannyi, az iránytól függetlenül.

6.2 Férőhelyek, üzemanyag-fogyasztás és gyártási év

Az árazás **objektivitásának alapfeltétele**, hogy a kritikus paraméterek — mint a férőhelyek száma, az üzemanyag-fogyasztás és a gyártási év — **ne szubjektív felhasználói bevitelre támaszkodjanak**, hanem validált, külső forrásból származó adatokból érkezzenek. Az adatintegritás biztosítása érdekében a rendszer egy többszintű, dinamikus adat lekérdezési folyamatot alkalmaz, amely a *RapidApiCarService* osztályon keresztül kommunikál a Rapid API gépjármű-adatbázisával.

Az alkalmazás indításakor a rendszer megvizsgálja a gépjármű márkák adatbázis tábláját, és ha szükséges lekéri a külső Rapid API -ban tárolt márkákat, és feltölti a saját adatbázisát ezzel. Ez csak az alkalmazás indításakor indul el, az *ApplicationReadyEvent* osztályra hallgatva. Az **entitások struktúrája** (márka, modell, generáció, típus) szorosan **illeszkedik a külső API** válaszformátumával. Ez teszi lehetővé, hogy a frontend legördülő listái dinamikusan frissüljenek: a felhasználó választása azonnali lekérdezést indít először az adatbázis, majd — ha ott nincs találat — a külső szolgáltatás irányába. Például, ha a felhasználó kiválasztja a Skoda gépjárműmárkát, akkor a modell legördülő lista automatikusan a Skodához tartozó opciókat tartalmazza. Ez azt jelenti, hogy az adatbázis kezdetben nem tárol kritikus információkat az adott autókról, csak a márkát. Ez egy fajta **„lazy loading”** mechanizmus, tehát csak akkor mentjük le az adatot, ha már valakinek szüksége volt rá. A lekérés után perzisztensen eltároljuk a rendszer adatbázisába, hogy később innen tudjuk „gyorsítótárazni”. Ezzel védjük a rendszerünket a külső API korlátaival szemben.

A gépjárművek pontos műszaki specifikációi — mint az ülőhelyek száma és a vegyes fogyasztás — csak a kalkulációs folyamat elindításakor kerülnek lekérésre és mentésre. A meghirdetett szabad ülőhelyek száma a kalkuláció előtt validálásra kerülnek, és érvénytelen

adat esetén ezt jelezzük a felhasználónak egy toast komponens formájában. Sajnos előfordulhat, hogy a külső szolgáltatás adatbázisa nem teljes, és hiányzik például a vegyes fogyasztás. Az alkalmazás éppen ebből az okból tárolja el az üzemanyag típusát is, ami alapján beállítja a fogyasztás európai átlagát. Ez benzines gépjárműnél 7.89 l/10km , dízelesnél pedig 6.88 l/100km . Abban az esetben, ha az üzemanyag típusa is hiányzik, vagy nincs a felismert opciók között, a lehetséges motortípusok fogyasztásának átlaga van használva [12]. A különböző motortípusok fogyasztásának átlaga a **6.1-es képlet** alapján számítható ki.

$$F_{avg} = \frac{(7.89 + 6.88 + 7.44 + 5.97 + 5.83 + 5.94)}{6}$$

6.1 Képlet, személygépkocsik átlagfogyasztása

A számításokhoz elengedhetetlen adatként szolgál a gépjármű kora, amely a kiválasztott generáció gyártási intervallumából becsülhető. Az algoritmus a kezdő és befejező gyártási év számtani átlagát vonja ki az idei évből. Abban az esetben, ha a befejező év nem definiált, a mostani évet használja az alkalmazás az átlag meghatározására.

6.3 Kilométerenkénti értékcsökkenés

Ahhoz, hogy igazságos, objektív árat tudjunk számolni, amivel mindkét fél jól jár, fontos, hogy figyelembe vegyük a gépjármű értékvesztését a megtett km alatt. A kilométerenkénti értékvesztés kiszámításához szükségünk van a gépjármű újkori árára és egy képletre, amely megbecsüli az autó jelenértékét. A dolgozatomban egy geometrikus értékcsökkenési modellt választottam, amely azt feltételezi, hogy az eszköz értéke egy állandó δ értékkel csökken az idő múlásával. A kutatások alapján az OECD (Gazdasági Együttműködési és Fejlesztési Szervezet) országokban, amelynek Magyarország is tagja, az éves átlagos értékcsökkenési ráta értéke 31% [13]. A geometrikus képlet, az éves átlagos értékcsökkenési ráta, az újkori ár és a jármű életkora alapján közelíti a jármű tárgyévre számolt értékét.

$$P_t = e^{-\delta t} * P_{t0}$$

6.2 Képlet, t éves gépjármű jelenértéke

A **6.2-es képletben** a P_t a t-edik éves gépjármű ára, δ az átlagos értékcsökkenés, P_{t0} pedig az eszköz újkori ára. Ezzel a képlettel kiszámolható a gépjármű idei és következő évi ára. Ennek különbsége az eszköz várható értékcsökkenése adott tárgyévre. Átlagosan 10 800 km-t tesznek

meg az Európai Unió sofőrei évente [14]. Az idei árból kivonva a következő évi árat és a különbséget elosztva az átlagosan évente megtett kilométerrel, megkapjuk a kilométerenkénti értékvesztést, ezt a számítást a 6.3-as képlet ábrázolja.

$$\frac{\text{Értékcsökkenés}}{km} = \frac{P_t - P_{t+1}}{\text{éves } km}$$

6.3 Képlet, kilométerenkénti értékcsökkenés

6.4 Végző kalkuláció

Egy adat híján, minden a rendelkezésünkre áll ahhoz, hogy kiszámítsuk az egy utasra jutó árat. A hiányzó adat, az **üzemanyag ára**, amire az alkalmazás a dokumentáció írása idejében országos szinten maximalizált értéket használja. Ez az érték **615 Ft**. Az idáig felsorolt komponensekből így áll össze az egy főre jutó útdíj egészre kerekítve:

$$K = \frac{d * \left(c + \frac{f * p}{100} \right)}{n}$$

6.4 Képlet, egy főre jutó útiköltség

A **6.4-es képlet** a végző, egy főre jutó költség értékét adja eredményül. A képletben a rövidítésként használt változók jelentése a következő:

- *K*: Egy főre jutó kalkulált útiköltség
- *c*: Kilométerenkénti értékvesztés
- *d*: Az út hossza kilométerben
- *f*: A gépjármű vegyes fogyasztása (l/100km)
- *p*: Az üzemanyag egységára
- *n*: Férőhelyek száma

7 Piaci összehasonlítás

Magyarország legmeghatározóbb telekocsi szolgáltatója az Oszkár Telekocsi [15]. A szolgáltató weboldalán publikusan elérhető hirdetési alapján életszerű összehasonlítást lehet végezni a jelen dolgozatban bemutatott alkalmazással. A 7.1 táblázat a **Szeged** és **Budapest** közötti útvonal egy főre jutó árait mutatja be a két rendszer viszonylatában:

Gépjármű Márka	Modell	Évjárat	Kapacitás (fő)	Ár (Oszkár)	Ár (Ride Sharing App)
Opel	Vivaro	2019	8	3 999 Ft	936 Ft
Toyota	Auris	2018	4	3 500 Ft	1 176 Ft
Volkswagen	Transporter T5	2018	8	4 499 Ft	793 Ft
Seat	Leon	2015	4	3 000 Ft	1 680 Ft
Toyota	Avensis	2015	4	3 000 Ft	1 393 Ft

7.1 Táblázat, egy főre jutó árak összehasonlítása

A 7.1-es táblázaton jól látható, hogy az egy főre jutó **árak lényegesen magasabbak** az Oszkár Telekocsi oldalon, mint amennyi a becsült költsége egy ilyen utazásnak. Elmondható, hogy az alkalmazás **objektíven** és **egyenletesen** osztja el az árat az utazó személyek között. Ezzel motiválva a platform felhasználóit a fuvar költségeinek megosztására.

Forrás: A táblázat adatai egyéni kutatás eredménye. Az Oszkár Telekocsi és a dolgozathoz készített Ride Sharing App webalkalmazás árait hasonlítja össze. Az adatok 2026.04.23-án lettek begyűjtve, és az applikáció 615 Ft-os üzemanyag árral számolt.

8 Befejezés

Az alkalmazás **elsődleges célja a profitmaximalizálás visszaszorítása** a telekocsizás terén, és ezzel a fuvarmegosztásra, mint szolgáltatásra való ösztönzés. A dolgozat legkritikusabb elemének — az árkalkulációnak — eredményességét egyértelműen bizonyítja a hetedik fejezetben bemutatott ár-összehasonlító táblázat.

A szoftver sikerességét nem csak a számadatok mutatják jól, hanem a kód komplexitása és minősége. A **websocket** technológiának köszönhetően a rendszer képes **késleltetés nélküli üzenetek** küldésére, ami elengedhetetlen a felek közötti bizalomépítés érdekében.

Az alkalmazás sikeresen **integrál három külső szolgáltatást**, ezek az Open WeatherMap, Open RouteService és RapidAPI. Mindez felszabadítja a felhasználót a manuális adatbeviteltől, és számolásoktól. Emellett hiteles, validált adatok megadására kényszerít minden sofőrt.

Az **optimalizált adatlekérések**, amelyek lapozási és intelligens szűrési technikák által valósultak meg, elősegítik az alkalmazás **gyors válaszát**, és **megelőzik** a kliens vagy szerver **túlterhelését**.

Az **értékelési rendszernek** köszönhetően, könnyen és gyorsan választhat partnert a felhasználó. A rendszer **ellehetetleníti a tisztességtelen** személyeket, de **előtérbe helyezi a becsületes** sofőröket és utasokat. Ez növeli az emberekben az online rendszer felé fordított bizalmat, és kedvezően hat a fuvarok költségeinek megosztására.

A **Keycloak** integrációja **JWT tokenek** által lehetővé tette, hogy minden felhasználó csak a saját erőforrásához férjen hozzá, és máshoz tartozó adatokat ne tudjon változtatni. Így az értékelések és utazások nem hamisíthatók.

A **Docker konténerizáció** révén a platform teljesen operációs rendszer független, így könnyedén bármely szervergépen, vagy felhőalapú környezetben minimális konfigurációval üzembe helyezhető.

9 Irodalomjegyzék

1. **Ridesharing services and urban transport CO2 emissions: Simulation-based evidence from 247 cities:** <https://www.sciencedirect.com/science/article/abs/pii/S1361920921002224?via%3Dihub>
Utolsó megtekintés dátuma: 2026.04.23.
2. **Osztogatnak vagy fosztogatnak? A sharing economy térnyerése:** https://www.pwc.com/hu/hu/kiadvanyok/assets/pdf/sharing_economy.pdf
Utolsó megtekintés dátuma: 2026.04.23.
3. **Proxying to a backend server:** <https://v17.angular.io/guide/build#proxying-to-a-backend-server>
Utolsó megtekintés dátuma: 2026.05.10.
4. **Open Weather Map Geocoding API:** <https://openweathermap.org/api/geocoding-api>
Utolsó megtekintés dátuma: 2026.05.10.
5. **Open Route Service — Matrix service:** <https://openrouteservice.org/dev/#/api-docs/v2/matrix/{profile}/post>
Utolsó megtekintés dátuma: 2026.05.10.
6. **Rapid API car specs:** <https://rapidapi.com/alekivanovski96-O1vKHrFskQm/api/car-specs>
Utolsó megtekintés dátuma: 2026.05.10.
7. **Keycloak:** <https://www.keycloak.org/>
Utolsó megtekintés dátuma: 2026.05.10.
8. **OAuth 2.0 Resource Server JWT:** <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>
Utolsó megtekintés dátuma: 2026.05.10.
9. **Using STOMPJS:** <https://stomp-js.github.io/guide/stompjs/using-stompjs-v5.html>
Utolsó megtekintés dátuma: 2026.05.10.
10. **Using websocket to build an interactive web application:** <https://spring.io/guides/gs/messaging-stomp-websocket>
Utolsó megtekintés dátuma: 2026.05.10.
11. **Why the worst day of the week is the best day to send emails:** <https://customer.io/learn/lifecycle-marketing/email-sending-schedule>
Utolsó megtekintés dátuma: 2026.04.23.
12. **A Bizottság jelentése az (EU) 2019/631 rendelet 12. cikkének (3) bekezdése alapján a személygépkocsik és a könnyű haszongépjárművek valós vezetési feltételek mellett mért széndioxid-kibocsátási különbségének alakulásáról az (EU) 2021/392 bizottsági végrehajtási rendelet 12. cikkében említett, a valós vezetési feltételek melletti anonimizált és összesített adatkészletek bemutatásával:** <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A52024DC0122>
Utolsó megtekintés dátuma: 2026.04.23.
13. **On the depreciation of automobiles: An international comparison:** https://cercetareservicii.ase.ro/resurse/Documente/On_the_depreciation_of_automobiles_An_international_comparison.pdf
Utolsó megtekintés dátuma: 2026.04.23.
14. **Change in distance travelled by car:** <https://www.odyssee-mure.eu/publications/efficiency-by-sector/transport/distance-travelled-by-car.html>
Utolsó megtekintés dátuma: 2026.04.23.
15. **Oszkár telekocsi:** <https://www.oszkar.com/>
Utolsó megtekintés dátuma: 2026.05.10.

10 Nyilatkozat

Alulírott programtervező infomatikus BSc szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztési Tanszékén készítettem, programtervező infomatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

2026. április. 22.

11 Köszönetnyilvánítás

Ezúton szeretném kifejezni hálámat a témavezetőmnek, Dr. Pengő Editnek, aki szakmai tanácsadásaival és útmutatásaival végig kísérte a szakdolgozatom elkészítését.

Külön köszönöm a rendszeres konzultációkat, valamint a szakmai megbeszélések során építő jellegű észrevételeit. Köszönöm, hogy rugalmasan és türelmesen állt a konzultációkhoz, és időbeosztásomnak megfelelően sikerült levezényelni a meetingeket.